

Exécutive Master Intelligence artificielle & science des données
Université Dauphine - PSL

Module - Deep Learning
Rapport de projet

Réseaux de Neurones Profonds pour le Jeu de Go

Implémentation et Diagnostic de Convergence

Rapport présenté par

- Malik CHETTIH
- Henri BALAMOU

Table des matières

1. Introduction	4
1.1 Contexte et Motivation.....	4
1.2 Évolution des techniques d'IA pour le Go :	4
1.2.1 Phase Monte Carlo Tree Search (MCTS).....	4
1.2.2 Phase Réseaux de Neurones Convolutifs	4
1.2.3 Révolution AlphaGo : Deep Learning + MCTS + Apprentissage par Renforcement.....	5
1.2 Problématique.....	5
1.2.1 Prédiction de Politique (Policy Prediction)	5
1.2.2 Évaluation de Position (Value Prediction)	6
1.2.3 Synergie Policy-Value	6
1.3 Références	7
2. Données et préparations.....	8
2.1 Source de données	8
2.2.1 Représentations des données	8
2.2.1.1 Structure des tenseurs d'entrée.....	9
Architecture des plans d'entrée.....	9
2.2.1.2 Format des données de sortie.....	9
Politique de jeu (Policy Head).....	9
Évaluation de position (Value Head)	10
2.3 Librairie Golois.....	10
2.3.1 Fonction getBatch() : Pipeline d'entraînement	10
2.3.2 Fonction getValidation() : Pipeline de validation	11
2.3.3 Optimisations et robustesse	11
2.4 Augmentation des données	11
2.4.1 Symétries géométriques du jeu de Go.....	11
2.4.2 Implémentation dans la bibliothèque Golois.....	12
2.4.3 Stratégie d'augmentation intensive sur batch	13
3. Architectures et choix techniques.....	15
3.1 Principes généraux.....	15
3.2 Choix d'Architectures.....	15
3.3 Organisation hiérarchique des classes	16
3.3.1 Analyse détaillée de la méthode build()	17
3.4 Spécifique par modèles	18
3.4.1 Modèle ResNet adapté au Go.....	18
3.4.2 Modèle MobileNet adapté au Go	20
3.4.3 Modèle MixNet adapté au Go	23
4. Procédure d'entraînements.....	25
4.1 Configuration et Initialisation.....	25
4.1.1 Section Expérimentation et Monitoring	26
4.1.2 Section Architecture du Modèle.....	26
4.1.3 Section Entraînement et Optimisation.....	27
4.1.3 Section Augmentation de Données	27
4.2 Optimiseur AdamW.....	28

4.3	Optimisation et Scheduling du Learning Rate	29
4.3.1	Cosine Decay avec Redémarrages	29
4.3.2	Adaptation Dynamique aux Modes d'Augmentation	30
4.4	Fonction d'Activation Swish.....	30
5	Monitoring	32
5.1	Architecture de Monitoring Intégrée	32
5.2	Gestion Robuste des Métriques Hétérogènes.....	32
5.3	Initialisation et Configuration WandB.....	33
5.4	Logging des Informations Architecturales	33
5.5	Gestion Avancée des Métriques d'Époque.....	34
5.6	Diagnostics de Gradients et Détection d'Anomalies.....	36
5.7	Sauvegarde Conditionnelle et Gestion des Artifacts.....	36
5.8	Synthèse Statistique et Analyse de Convergence.....	37
6	Framework de Tests.....	38
7	Tests Expérimentaux et Diagnostics.....	40
7.1	Démarche expérimental.....	40
7.2	Tests Expérimentaux.....	40
7.2.1	Test différentes architectures de modèles	40
7.2.2	Test de la profondeur du réseau	40
7.2.2	Test Learning rate optimal.....	41
7.2.2	Test de la dimension de la taille des batches d'entraînement	43
7.2.3	Test MixNet.....	44
7.2.4	Test du learning rate et politique d'adaptation.....	44
7.2.5	Test du Cosine Anneling avec redémarrage.....	44
7.2.6	Test de la dimension des données entraînées pour une epoch.....	45
8	Conclusions.....	46
9	Annexes	47

1. Introduction

1.1 Contexte et Motivation

Le jeu de Go représente l'un des défis les plus complexes en intelligence artificielle. Le jeu de go est un jeu combinatoire à deux joueurs qui se pratique sur une grille carrée de taille 19x19. La combinatoire du jeu de go explose lorsque la taille de la grille augmente, le go est classé parmi les problèmes NP-difficiles, et même PSPACE-difficiles. Avec un nombre de parties raisonnables possibles estimé à 10^{600} pour un goban 19x19, comparé à 10^{120} pour les échecs, sa complexité combinatoire dépasse largement celle de tous les autres jeux de stratégie.

1.2 Évolution des techniques d'IA pour le Go :

Le jeu de go était la cible des programmeurs depuis la défaite de Kasparov contre Deep Blue aux échecs. Les méthodes traditionnelles (Branch and Bound) se montraient insuffisantes pour le go, car il manquait une méthode d'évaluation efficace de la position permettant de réduire la combinatoire.

Trois innovations majeures ont successivement révolutionné l'approche :

1.2.1 Phase Monte Carlo Tree Search (MCTS)

L'obstacle de l'évaluation de position a été levé par l'introduction de méthodes stochastiques : des parties aléatoires (playouts) sont générées à partir de la position qu'on veut évaluer, et permettent de former un indicateur statistique (moyenne des scores de ces parties aléatoires, en comptant 1 pour une victoire et 0 pour une défaite). Ce mécanisme d'évaluation est intégré à une méthode d'exploration arborescente nommée MTCS pour Monte Carlo Tree Search.

Les résultats des actions ayant déjà été effectuées au cours des playouts précédents sont mémorisés. Les actions ayant des scores statistiques élevées sont privilégiées par rapport aux actions ayant de moins bons scores. La recherche Monte-Carlo est améliorée par l'utilisation de patterns dont les poids sont appris à partir de parties de joueurs forts. Ces patterns permettent de sélectionner les meilleurs coups dans l'arbre et d'améliorer les parties aléatoires.

Limitations : Ces idées ont permis de produire des programmes de go de niveau professionnel sur plateau 9x9, mais qui restaient encore en deçà du niveau des joueurs professionnels pour la dimension standard 19x19.

1.2.2 Phase Réseaux de Neurones Convolutifs

L'introduction des réseaux de neurones convolutifs marque une seconde révolution. Une couche convolutionnelle consiste à passer un filtre 3x3 sur tout le plan d'entrée. C'est une technique surtout utilisée en reconnaissance d'image et elle s'applique bien au jeu de Go qui peut être vu comme une image en deux dimensions. Le rectifieur linéaire est indispensable à l'apprentissage profond : son rôle est de redonner de la force au signal pour pouvoir modifier les couches profondes lors de l'apprentissage.

Avancée : Les premiers réseaux convolutifs permettent d'atteindre des taux de prévision du meilleur coup autour de 44.4%, une amélioration significative par rapport aux méthodes précédentes.

1.2.3 Révolution AlphaGo : Deep Learning + MCTS + Apprentissage par Renforcement

La société Google réunit une équipe qui conçoit AlphaGo, combinant pour la première fois les trois approches de manière synergique. La force d'AlphaGo est d'utiliser des réseaux de neurones profonds pour apprendre à trouver de bons coups au jeu de Go et pour apprendre à évaluer une position.

Architecture technique : L'apprentissage est réparti en trois phases, réalisées aux moyens de réseaux de neurones ayant la même structure en 13 couches. Chaque couche est convolutionnelle avec de 128 à 384 filtres suivi d'un rectifieur linéaire.

Les trois phases d'apprentissage d'Alpha GO :

1. **Apprentissage supervisé** : Apprendre à retrouver des coups de joueurs experts à partir de 30 millions de positions. Résultat : 57.0% de taux de prévision du meilleur coup (vs 44.4% précédemment).
2. **Apprentissage par renforcement** : Le programme joue des parties contre différentes versions de lui-même et apprend les résultats des parties ainsi jouées pour s'améliorer. Si les récompenses sont positives le comportement est renforcé, sinon il est diminué. Résultat : 80% de victoires contre le réseau original.
3. **Réseau évaluateur** : Apprendre à prévoir l'issue d'une partie à l'aide d'un réseau qui prend en entrée une position de Go et estime en sortie la probabilité de gain associée. 30 millions de positions différentes sont générées à partir de parties jouées contre lui-même.

Innovation clé : Les différents réseaux de neurones sont finalement combinés dans un programme qui fait de la recherche Monte-Carlo en utilisant les réseaux qui choisissent les coups pour biaiser les parties aléatoires et le réseau évaluateur pour biaiser les statistiques sur les parties aléatoires.

Performances : Le programme bat les meilleurs programmes de Go 99.8% du temps et remporte 5-0 contre Fan Hui, premier joueur professionnel battu sans handicap sur plateau 19×19. Les évolutions successives (AlphaGo Zero, AlphaZero) ont ensuite démontré la puissance de l'apprentissage par renforcement pur, sans données humaines initiales.

1.2 Problématique

La problématique qui se pose à nous lors d'une partie de jeu de Go est double. A chaque coup joué nous avons besoin d'évaluer sa pertinence par rapport aux autres coup possible. Nous avons aussi besoin d'évaluer l'impact de ce coup dans la partie jouée. Nous avons besoin d'évaluer à partir de ce coup quelle est notre chance finale de gagner la partie.

1.2.1 Prédiction de Politique (Policy Prediction)

L'objectif étant de déterminer la probabilité qu'un coup donné soit joué dans une position donnée par un joueur expert.

Nous pouvons formaliser cela par :

Soit une position P sur le plateau 19×19 , la fonction policy $\pi(a|P)$ doit estimer la probabilité que l'action A (placement d'une pierre sur l'intersection A) soit choisie. Cette distribution de probabilités sur les 361 intersections possibles guide la sélection des coups les plus prometteurs.

On relevera dans ce contexte de prédiction, les défis suivants :

- **La sparsité** : Sur 361 coups possibles, seuls quelques-uns sont réellement viables (typiquement 20-50)
- **Dépendance contextuelle** : Un même coup peut être excellent ou catastrophique selon le contexte global
- **Apprentissage des motifs** : Nécessite de capturer les patterns tactiques et stratégiques à différentes échelles

1.2.2 Évaluation de Position (Value Prediction)

Objectif : Estimer la valeur d'une position pour le joueur à jouer, c'est-à-dire prédire l'issue probable de la partie à partir de cette position.

Formalisation : Soit une position P , la fonction value $V(P) \in [0,1]$ estime la probabilité de victoire du joueur à jouer. Cette évaluation remplace les simulations Monte Carlo coûteuses en temps de calcul.

Défis techniques :

- **Horizon temporel** : Une position peut sembler favorable à court terme mais désastreuse à long terme
- **Compensation** : Les avantages territoriaux vs. l'influence doivent être équilibrés
- **Phases de jeu** : L'évaluation diffère radicalement entre ouverture, milieu de partie et fin de partie
- **Subtilité positionnelle** : Certains avantages (thickness, aji) sont difficiles à quantifier

Applications : La value remplace l'évaluation par playouts aléatoires dans MCTS, permettant une estimation plus précise et plus rapide de la qualité des positions feuilles.

1.2.3 Synergie Policy-Value

Complémentarité : Ces deux fonctions travaillent en synergie dans l'algorithme de recherche :

- La **policy** guide l'exploration vers les coups les plus plausibles
- La **value** évalue les positions résultantes sans simulation complète
- Ensemble, elles permettent une recherche efficace dans un espace combinatoire gigantesque

Défis d'apprentissage joint : L'entraînement simultané de ces deux fonctions sur un réseau partagé pose des défis spécifiques :

- **Conflits d'objectifs** : Les gradients des deux têtes peuvent s'opposer
- **Pondération des pertes** : Équilibrer l'importance relative des deux tâches
- **Convergence** : Assurer que les deux objectifs convergent harmonieusement

Un défi technique majeur réside dans la convergence des réseaux de neurones profonds, particulièrement sensible aux problèmes de gradients explosifs ou évanescents lors de l'optimisation jointe de ces deux objectifs distincts.

1.3 Références

Dans le cadre de ce projet, les références suivantes ont été utilisées :

- [1] "Deep Learning with Python", Francois Chollet. Manning, 2020. [book](#).
- [2] [Keras](#) and [Tensorflow](#)
- [3] "Residual Networks for Computer Go", Tristan Cazenave. IEEE Transactions on Games, Vol. 10 (1), pp 107-110, March 2018. [resnet.pdf](#).
- [4] "Mastering the game of Go without human knowledge", David Silver et al. Nature 2017. [AlphaGoZero](#).
- [5] "Spatial Average Pooling for Computer Go", Tristan Cazenave. CGW at IJCAI 2018. [sap.pdf](#).
- [6] "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play", David Silver et al. Science 2018. [AlphaZero](#)
- [7] "Accelerating Self-Play Learning in Go", David J. Wu. AAAI RLG 2020. [accelerating.pdf](#)
- [8] "Polygames: Improved Zero Learning", Tristan Cazenave et al. ICGA Journal 2020. [polygames.pdf](#)
- [9] "Mobile Networks for Computer Go", Tristan Cazenave. IEEE Transactions on Games, 2021. [MobileNetworksForComputerGo.pdf](#)
- [10] "Improving Model and Search for Computer Go", Tristan Cazenave. IEEE Conference on Games 2021. [ImprovingModelAndSearchForComputerGo.pdf](#)
- [11] "Cosine Annealing, Mixnet and Swish Activation for Computer Go", Tristan Cazenave, Julien Sentuc, Mathurin Videau. Advances in Computer Games 2021. [CosineAnnealingMixnetAndSwishActivationForComputerGo.pdf](#)

2. Données et préparations

2.1 Source de données

L'entraînement de réseaux de neurones pour le jeu de Go nécessite des données de haute qualité pour atteindre un niveau de jeu compétitif. Dans ce projet, nous utilisons le dataset KataGo, considéré actuellement comme l'une des meilleures sources de données d'entraînement disponibles pour l'apprentissage automatique au Go.

KataGo représente l'état de l'art des moteurs de Go, surpassant en force de jeu des programmes reconnus comme ELF/OpenGo et Leela Zero. Ce programme utilise des techniques d'apprentissage par renforcement profond combinées à la recherche Monte-Carlo (MCTS), similaires à celles développées par AlphaZero, mais avec des optimisations avancées qui permettent une convergence plus rapide vers un niveau surhumain.

Notre dataset se compose de **364,903 parties** auto-jouées par KataGo, extraites des fichiers SGF publiés en 2020. Ces parties présentent plusieurs avantages significatifs par rapport aux datasets traditionnels :

Qualité supérieure : Contrairement aux datasets basés sur des parties humaines (même de niveau professionnel), nos données proviennent d'un agent ayant déjà atteint un niveau surhumain. Chaque coup et chaque évaluation de position reflètent une compréhension optimale du jeu, dépassant les capacités des meilleurs joueurs humains.

Sélection rigoureuse : Les parties retenues sont exclusivement jouées sur goban 19x19 avec un komi compris entre 5.5 et 7.5, garantissant une cohérence dans les conditions de jeu standard. Cette sélection élimine les variations exotiques qui pourraient introduire du bruit dans l'apprentissage.

Optimisations d'entraînement : KataGo utilise des stratégies d'optimisation avancées, notamment l'utilisation d'un nombre réduit de playouts pour la majorité des coups, permettant de générer plus de données d'évaluation en moins de temps. Cette approche, combinée à des cibles d'entraînement supplémentaires pour la régularisation, produit des données d'une richesse exceptionnelle.

Format d'entraînement : Chaque position est accompagnée des probabilités de politique prédites par KataGo et des évaluations de valeur calculées par son moteur MCTS. Cette double annotation (politique + valeur) permet l'entraînement simultané des deux têtes du réseau, optimisant à la fois la sélection de coups et l'évaluation de positions.

L'utilisation de ce dataset de référence constitue un fondement solide pour notre projet, nous permettant de nous concentrer sur l'optimisation architecturale et les techniques d'entraînement tout en bénéficiant de données d'une qualité inégalée dans le domaine du Go informatique.

2.2.1 Représentations des données

La représentation des positions de Go sous forme tensorielle constitue un défi majeur dans l'adaptation des réseaux de neurones convolutionnels au jeu de Go. Notre implémentation utilise un encodage sophistiqué qui transforme chaque position de jeu en un tenseur 4D de dimensions **19×19×31**, où chaque plan (channel) capture une caractéristique spécifique de la position.

2.2.1.1 Structure des tenseurs d'entrée

Architecture des plans d'entrée

Les **31 plans** se décomposent selon plusieurs catégories d'informations organisées séquentiellement :

Plan du joueur actuel (1 plan) : Le plan 0 indique le joueur à jouer (1.0 si Noir doit jouer, 0.0 si c'est au tour des Blancs).

Analyse tactique des échelles (2 plans) : Les plans 1-2 encodent les informations d'échelles (ladders). Le plan 1 marque les pierres perdues en échelle (isLostLadder), le plan 2 indique si des groupes adjacents sont en échelle (adjIsLostLadder).

Plans d'échelles avancées (2 plans) : Les plans 3-4 fournissent une analyse plus sophistiquée avec isInLadder et adjIsInLadder, détectant les situations d'échelles complexes.

Libertés des groupes (6 plans) : Les plans 5-10 (MaxLib = 6) encodent le nombre de libertés de chaque groupe sur le goban (1, 2, 3, 4, 5, ou ≥ 6 libertés). Cette information est cruciale pour évaluer la force et la survie des groupes.

Historique des positions (10 plans) : Les plans 11-20 ($2 * \text{Planes} = 10$ avec Planes = 5) conservent l'historique des 5 dernières positions. Chaque position historique utilise 2 plans : un pour les pierres du joueur actuel, un pour les pierres de l'adversaire.

Plan de valeur (1 plan) : Le plan 21 ($2 * \text{Planes} + 2$) contient parfois la valeur de la position, utilisée comme information auxiliaire pendant l'entraînement.

Informations de Komi (10 plans) : Les plans 22-31 encodent la valeur du komi (0.5 à 9.5) sous forme one-hot. Un seul plan est activé (valeur 1.0) selon le komi de la partie, tous les autres restent à 0.0.

2.2.1.2 Format des données de sortie

Politique de jeu (Policy Head)

La sortie politique est représentée par un vecteur de **361 éléments** correspondant aux 361 intersections possibles du goban 19×19. Chaque élément contient la probabilité que KataGo joue le coup correspondant dans la position donnée. Cette représentation utilise un encodage "one-hot" où seul le coup effectivement joué par KataGo reçoit la valeur 1.0, tous les autres coups recevant 0.0.

Le coup "pass" (passer son tour) est encodé spécialement avec l'index 361, permettant au réseau d'apprendre quand il est approprié de passer plutôt que de jouer un coup sur le goban.

Évaluation de position (Value Head)

La valeur de chaque position est encodée comme un scalaire flottant représentant l'évaluation MCTS de KataGo. Cette valeur, comprise entre -1.0 et +1.0, indique la probabilité de victoire estimée pour le joueur actuel (-1.0 = défaite certaine, +1.0 = victoire certaine, 0.0 = position équilibrée).

2.3 Librairie Golois

La bibliothèque Golois constitue l'interface principale entre les données KataGo et l'entraînement des réseaux de neurones. Implémentée en C++ avec des bindings Python via PyBind11, elle optimise les performances critiques du préprocessing des données tout en s'intégrant naturellement à l'écosystème TensorFlow/Keras.

2.3.1 Fonction `getBatch()` : Pipeline d'entraînement

La fonction **`getBatch()`** orchestre la génération dynamique des mini-batches d'entraînement selon un processus sophistiqué :

Chargement et indexation des données : Au premier appel, la fonction charge le fichier `games.data` contenant les 364,903 parties KataGo. Un système d'indexation (`positionSGF`) référence chaque position jouable, excluant automatiquement les coups "pass" (move 361) et les positions de validation.

Sélection pseudo-aléatoire : Pour chaque échantillon du batch, la fonction sélectionne une position via $(\text{index} + i) \% \text{nbPositionsSGF}$, garantissant une exploration déterministe mais variée du dataset. Cette approche cyclique évite la redondance excessive tout en maintenant la stochasticité nécessaire à l'entraînement.

Reconstruction de position : La position sélectionnée est reconstituée en rejouant séquentiellement tous les coups depuis le début de partie via `playColor()`. Cette reconstruction dynamique évite le stockage de millions de positions pré-calculées, optimisant l'usage mémoire.

Encodage tensoriel adaptatif : L'appel à `encodeSymKomi()` effectue simultanément l'encodage des 31 plans et l'application d'une transformation géométrique aléatoire (parmi les 8 symétries possibles). Cette fusion optimise les performances en évitant les copies multiples de tenseurs.

Génération des labels : La politique est encodée en one-hot avec $\pi(i, \text{move}) = 1.0$ pour le coup joué par KataGo, tous les autres restant à 0.0. La valeur MCTS (`proGame[game][move].val`) est directement assignée comme label de régression.

Tenseur de fin de partie : Un tenseur auxiliaire (`end`) encode l'état final hypothétique du goban en simulant la fin de partie depuis la position courante. Cette information aide le réseau à comprendre les conséquences à long terme des coups.

2.3.2 Fonction `getValidation()` : Pipeline de validation

La fonction `getValidation()` implémente un mécanisme de validation rigoureux garantissant l'intégrité de l'évaluation :

Génération déterministe du set de validation : Utilisant une graine fixe (`srand(0)`), la fonction génère un ensemble de validation reproductible. Ce déterminisme est crucial pour comparer objectivement différentes architectures ou hyperparamètres.

Séparation stricte train/validation : Le système marque comme `validationGame[game] = true` toutes les parties utilisées pour la validation. Ces parties et toutes leurs positions sont définitivement exclues de l'entraînement, éliminant tout risque de fuite de données (data leakage).

Stratégie d'échantillonnage : Une position aléatoire est sélectionnée par partie de validation, créant un ensemble diversifié représentatif de différents stades de jeu. Cette approche évite les biais temporels qui pourraient survenir en sélectionnant systématiquement les mêmes phases de partie.

Cache persistant : Les indices de validation sont sauvegardés dans `validation.data`, permettant la reproductibilité entre exécutions. Ce cache accélère les lancements ultérieurs tout en maintenant la cohérence des évaluations.

Encodage symétrique cohérent : Contrairement à `getBatch()`, la validation applique une transformation géométrique basée sur l'index (`pos % 8`) plutôt qu'aléatoire, assurant une évaluation stable et reproductible.

2.3.3 Optimisations et robustesse

Gestion mémoire intelligente : La bibliothèque alloue dynamiquement les structures selon la taille des batches, s'adaptant aux contraintes matérielles sans gaspillage.

Filtrage automatique : Les positions illégales, les coups pass, et les parties corrompues sont automatiquement détectés et exclus, garantissant la qualité des données d'entraînement.

Interface thread-safe : Bien que non explicitement parallélisée, l'architecture permet l'extension vers le multiprocessing pour l'entraînement distribué.

La dualité entre `getBatch()` et `getValidation()` permet un entraînement efficace et une évaluation reproductible des modèles.

2.4 Augmentation des données

2.4.1 Symétries géométriques du jeu de Go

Le jeu de Go possède une propriété géométrique fondamentale : l'équivalence de toutes les transformations isométriques du goban. Cette invariance naturelle découle de l'homogénéité du plateau et de l'absence d'orientation privilégiée des règles. Mathématiquement, le groupe de

symétries du carré génère 8 transformations distinctes qui préservent parfaitement la validité et la valeur stratégique de toute position.

Les 8 transformations canoniques

Identité (T0) : Position originale sans modification, servant de référence pour toutes les autres transformations.

Rotations :

- **T1 - Rotation 90° horaire :** $(x,y) \rightarrow (18-y, x)$ - Les pierres du bord gauche migrent vers le bord supérieur
- **T2 - Rotation 180° :** $(x,y) \rightarrow (18-x, 18-y)$ - Symétrie centrale, inversion complète du plateau
- **T3 - Rotation 270° horaire :** $(x,y) \rightarrow (y, 18-x)$ - Les pierres du bord supérieur migrent vers le bord gauche

Réflexions :

- **T4 - Miroir horizontal :** $(x,y) \rightarrow (x, 18-y)$ - Réflexion selon l'axe vertical central
- **T5 - Miroir vertical :** $(x,y) \rightarrow (18-x, y)$ - Réflexion selon l'axe horizontal central

Compositions :

- **T6 - Transposition diagonale :** $(x,y) \rightarrow (y, x)$ - Réflexion selon la diagonale principale
- **T7 - Transposition anti-diagonale :** $(x,y) \rightarrow (18-y, 18-x)$ - Réflexion selon la diagonale secondaire

Chaque transformation préserve rigoureusement les relations tactiques (libertés, captures, connexions) et les évaluations stratégiques, garantissant l'équivalence parfaite des positions transformées.

2.4.2 Implémentation dans la bibliothèque Golois

Gestion native des symétries

La bibliothèque Golois intègre nativement la gestion des symétries à deux niveaux complémentaires :

Au niveau de l'encodage : La fonction `encodeSymKomi()` applique dynamiquement une transformation aléatoire lors de la génération de chaque échantillon. Cette approche, héritée d'AlphaGo, distribue uniformément les symétries sur l'ensemble du dataset sans surcoût mémoire.

Table de correspondance des coups : Le système `sym[move][randomSym]` pré-calculé la correspondance de chaque intersection (0-360) vers ses 8 variants géométriques. Cette table optimise les transformations de politique en évitant les calculs géométriques répétitifs pendant l'entraînement.

Cohérence tenseur-politique : Chaque transformation appliquée au tenseur d'entrée (19×19×31) est automatiquement répercutée sur le vecteur de politique (361 éléments), maintenant la cohérence parfaite entre représentation spatiale et labels d'entraînement.

Intégration transparente

L'implémentation golois rend l'augmentation transparente pour l'utilisateur : chaque appel à `getBatch()` génère automatiquement des positions géométriquement variées sans modification du code d'entraînement. Cette transparence simplifie l'expérimentation tout en garantissant la diversité des données.

2.4.3 Stratégie d'augmentation intensive sur batch

Motivation de l'approche

Notre implémentation adopte une stratégie d'**augmentation intensive par batch** qui diffère des approches traditionnelles. Plutôt que d'appliquer une seule transformation aléatoire par échantillon, nous entraînons séquentiellement le réseau sur les 8 variants de chaque batch, multipliant efficacement par 8 la diversité des données d'entraînement.

Cette approche présente plusieurs avantages décisifs :

Couverture exhaustive : Chaque position du dataset est explorée sous tous ses angles géométriques durant l'entraînement, maximisant l'apprentissage de l'invariance rotationnelle et réflexive.

Stabilité d'apprentissage : L'exposition systématique aux 8 variants réduit la variance des gradients et stabilise la convergence, particulièrement bénéfique pour les architectures sensibles comme MobileNet.

Efficacité computationnelle : Les transformations s'effectuent sur place (in-place) sans allocation mémoire supplémentaire, évitant l'explosion de la consommation RAM qui résulterait d'une multiplication naive des données.

Algorithme d'implémentation

```
for epoch in range(epochs):
    # Génération du batch original via Golois
    golois.getBatch(input_data, policy, value, end, groups, offset)

    # Sauvegarde des données originales
    original_input = input_data.copy()
    original_policy = policy.copy()

    # Entraînement sur les 8 transformations
    for transform_id in range(8):
        # Restauration des données originales
        input_data[:] = original_input
        policy[:] = original_policy

        # Application de la transformation sur place
        if transform_id > 0: # T0 = identité
            apply_transformation_batch(input_data, policy, transform_id)

    # Entraînement sur cette variant
```

```
model.fit(input_data, {'policy': policy, 'value': value})
```

Optimisations mémoire

Transformations sur place : L'utilisation de `numpy.rot90()`, `numpy.flip()` et `numpy.transpose()` avec assignation directe (`array[:] = transformed`) évite la création de copies temporaires. Cette technique maintient l'empreinte mémoire constante indépendamment du nombre de transformations.

Réutilisation des buffers : Les tenseurs `input_data` et `policy` servent de buffers réutilisables, évitant les allocations/désallocations répétées qui fragmenteraient la mémoire.

Gestion adaptative des batches : Le système ajuste automatiquement le learning rate schedule pour compenser la multiplication par 8 du nombre d'itérations, maintenant la cohérence de l'optimisation.

Impact sur l'entraînement

Cette stratégie transforme fondamentalement la dynamique d'apprentissage :

Généralisation renforcée : L'exposition systématique aux variants géométriques force le réseau à apprendre des représentations invariantes, améliorant significativement la généralisation sur des positions inédites.

Régularisation implicite : L'augmentation intensive agit comme un régularisateur naturel, réduisant l'overfitting et améliorant la robustesse du modèle.

Accélération de convergence : Paradoxalement, bien que multipliant par 8 le nombre d'itérations par époque, cette approche accélère souvent la convergence vers un optimum de meilleure qualité.

L'implémentation de cette stratégie d'augmentation représente un compromis optimal entre performance computationnelle, efficacité mémoire et qualité d'apprentissage, particulièrement adapté aux contraintes du deep learning pour le jeu de Go.

3. Architectures et choix techniques

3.1 Principes généraux

Dans le cadre de notre projet, nous testerons plusieurs architectures comme les réseaux de types « Residual Network », « Mobile Network » et « MixNet Network ». Ces architectures modernes ont démontré leur efficacité dans le domaine du jeu de Go, avec notamment les Mobile Networks qui surpassent les Residual Networks classiques tant pour les réseaux petits que larges sur les datasets de niveau surhumain. L'introduction des blocs Squeeze and Excitation dans les Mobile Networks améliore encore les performances grâce à l'attention sur les canaux.

Notre implémentation de référence adopte une architecture convolutive simplifiée mais efficace, spécifiquement adaptée aux contraintes computationnelles du jeu de Go. Le réseau prend en entrée des tenseurs de dimensions (19, 19, 31), où les 31 plans d'entrée encodent l'état complet de la position : configuration actuelle du plateau, historique des coups, informations sur les libertés et les échelles (ladders).

Le réseau adopte une architecture à deux têtes d'AlphaGo avec des adaptations modernes. La tête policy utilise une convolution 1×1 sans biais suivie d'un aplatissement et d'une activation softmax pour produire une distribution de probabilités sur les 361 intersections possibles. La tête value applique également une convolution 1×1, puis une couche dense de 50 neurones avec activation ReLU, et finalement une sortie sigmoïde pour l'évaluation de position dans [0,1].

La fonction de perte combine deux composantes principales selon la formulation :

$$\text{Loss_totale} = \alpha \times \text{Loss_policy} + \beta \times \text{Loss_value}$$

Où :

- **Loss_policy** : Entropie croisée catégorielle pour optimiser la prédiction des coups
- **Loss_value** : Erreur quadratique moyenne (MSE) pour l'évaluation de position
- **Pondération** : $\alpha = 1.0$, $\beta = 1.0$ (optimisée expérimentalement)

Cette pondération privilégie légèrement l'apprentissage de la politique de jeu tout en maintenant une évaluation précise des positions, configuration qui s'est révélée optimale lors de nos expérimentations préliminaires.

3.2 Choix d'Architectures

Dans notre démarche expérimentale nous avons fait le choix de tester plusieurs types de modèles. Pour cela nous avons identifié trois types d'architectures que nous voulons comparer. Nous avons pris comme référence un modèle à base de connexion résiduelle, modèle **ResNet**. Ce type de modèle ayant été cité dans un certain nombre d'articles scientifique, il nous servira comme étalon et comparateur par rapport aux autres types de modèles testés [10].

Le deuxième modèle est un modèle de type **MobileNet**. Cité dans [9], ce type de modèle a été comparé et réputé plus performant que le modèle **ResNet** classique. Plusieurs versions de modèle **MobileNet** ont été testé et seront décrits dans la suite de cette section.

Le dernier modèle que nous testerons sera de type **MixNet**. Ce modèle, qui est cité dans [11], devrait être plus performant que les modèle de type **MobileNet**. Contrairement au model **MobileNet** qui

sont construit sur la base de couche convolutionnelles incluant une seule taille de filtre, par exemple filtre (3x3), les modèle de type **MixNet** permettent de combiner dans une seule couche plusieurs taille de filtres, par exemples (3x3) et (5x5). Le mélange de taille de filtre est sensé nous permettre de capter de façon plus efficace les patterns permettant d'identifier les meilleures positions dans le jeu de Go.

Nous avons fait le choix de nous concentrer sur des architecture/modèle construits à partir de réseaux de neurones à base de couches convolutionnelles. Ce type de couche, initialement utilisées dans la reconnaissance d'image ou de vidéos est aujourd'hui identifié comme très performante dans l'IA des jeux et tout particulièrement pour jeu de Go.

3.3 Organisation hiérarchique des classes

Notre implémentation adopte une architecture orientée objet rigoureuse basée sur le patron de conception Template Method. Cette organisation favorise la réutilisabilité du code, la maintenabilité et l'extensibilité vers de nouvelles architectures.

La classe **GoBaseNet** constitue le socle architectural commun aux trois modèles implémentés. Elle centralise les méthodes utilitaires et définit l'**interface standard** pour la construction des réseaux.

La classe **GoBaseNet** contient une méthode **build()** abstraite. Chaque classe dérivée doit implémenter sa propre méthode **build()**, garantissant une interface uniforme tout en permettant des architectures spécialisées.

```
def build(self, block_num=2, filters=32, factor=4, se=True,
          drop_out_rate=0.5, activation='swish'):

    inputs = Input(shape=self.shape)
    x = self._conv_block(inputs, filters, (1, 1), activation='relu')

    for i in range(block_num):
        x = self.get_iteration_block(x, filters, (3, 3), activation='relu')

    policy_head = self._conv_block(x, filters=1, kernel=(1, 1))
    policy_head = Flatten()(policy_head)
    policy_head = Activation('softmax', name='policy')(policy_head)

    value_head = GlobalAveragePooling2D()(x)
    value_head = Dense(50, kernel_regularizer=regularizers.l2(0.0001),
                      kernel_initializer='he_normal')(value_head)

    value_head = Activation('relu')(value_head)
    value_head = Dropout(drop_out_rate)(value_head)
    value_head = Dense(1, activation='sigmoid', name='value',
                      kernel_regularizer=regularizers.l2(0.0001),
                      kernel_initializer='he_normal')(value_head)

    model = keras.Model(inputs=inputs, outputs=[policy_head, value_head])
    return model
```


3.3.1 Analyse détaillée de la méthode build()

La **méthode build()** constitue le cœur architectural de nos modèles, orchestrant la construction séquentielle du réseau selon une structure invariante : **entrée → backbone → têtes de prédiction**. Cette approche standardisée garantit la cohérence tout en permettant des spécialisations architecturales.

```
def build(self, block_num=2, filters=32, factor=4, se=True,
          drop_out_rate=0.5, activation='swish'):
```

La méthode accepte six paramètres configurables qui déterminent les caractéristiques architecturales du réseau :

- **block_num** : Nombre de blocs répétés dans le backbone (profondeur)
- **filters** : Nombre de canaux de base (largeur du réseau)
- **factor** : Facteur d'expansion pour les blocs bottleneck (complexité interne)
- **se** : Activation du mécanisme Squeeze-and-Excitation (attention des canaux)
- **drop_out_rate** : Taux de dropout pour régularisation
- **activation** : Fonction d'activation ('relu', 'swish', etc.)

Phase d'initialisation

La construction débute par la définition de la couche d'entrée qui accepte des tenseurs de forme correspondant aux 31 plans d'encodage du goban. Cette entrée est immédiatement transformée par une convolution de projection qui réduit les 31 canaux d'entrée vers le nombre de canaux de travail spécifié par le paramètre **filters**. Cette convolution utilise un kernel de taille 1×1 pour effectuer une transformation linéaire pure sans mélange spatial, accompagnée d'une normalisation par batch et d'une activation ReLU. Cette étape cruciale établit l'espace de représentation interne du réseau et standardise les dimensions pour le traitement ultérieur.

Construction du backbone architectural

Le backbone se construit via une boucle itérative qui répète **block_num** fois l'application d'un bloc architectural spécialisé. Cette boucle constitue le cœur différenciateur entre les trois architectures implémentées. Pour MobileNet, chaque itération applique un bloc bottleneck utilisant des convolutions séparables en profondeur avec expansion et projection, optimisant l'efficacité computationnelle. MixNet remplace la convolution séparable standard par une convolution mixte combinant des kernels 3×3 et 5×5 en parallèle, capturant simultanément des patterns à différentes échelles spatiales. ResNet utilise des blocs résiduels classiques avec double convolution standard et connexion résiduelle directe, privilégiant l'expressivité représentationnelle.

Chaque bloc maintient les dimensions spatiales du tenseur à 19×19 tout en raffinant progressivement les représentations internes. Les mécanismes de Squeeze-and-Excitation peuvent être activés pour

introduire un système d'attention sur les canaux, recalibrant l'importance relative des différentes features. Les connexions résiduelles, lorsqu'elles sont applicables, facilitent la propagation des gradients et permettent l'apprentissage de transformations plus profondes.

Création des têtes de prédiction

La phase finale construit deux têtes de prédiction spécialisées partageant le même backbone. La tête de politique transforme les features du backbone en prédictions de coups via une convolution de réduction vers un seul canal, suivie d'un aplatissement spatial et d'une normalisation softmax. Cette architecture produit 361 sorties correspondant aux intersections du goban, avec une distribution de probabilité normalisée.

La tête de valeur évalue la position globale en agrégeant spatialement les features via un pooling global moyen, puis en les transformant à travers une couche dense intermédiaire de 50 neurones et une sortie finale sigmoïde. Cette architecture en entonnoir résume efficacement l'information spatiale distribuée en une évaluation scalaire de la position. Un mécanisme de dropout peut être appliqué avant la sortie finale pour régulariser l'apprentissage, bien que certaines architectures comme MixNet puissent l'omettre selon les besoins de stabilité.

Assemblage final

La méthode conclut en assemblant ces composants en un modèle Keras unifié avec une entrée unique et deux sorties nommées. Cette architecture dual-head permet l'entraînement joint des deux tâches, favorisant le partage de représentations et la régularisation mutuelle entre la prédiction de coups et l'évaluation de positions.

3.4 Spécifique par modèles

3.4.1 Modèle ResNet adapté au Go

Les réseaux de neurones ResNet (Residual Networks) sont une famille d'architectures de deep learning introduites en 2015, conçues pour entraîner des réseaux très profonds grâce à l'utilisation de **blocs résiduels**. Ces blocs ajoutent une connexion de saut (Skip Connection) qui permet au réseau d'apprendre une fonction résiduelle plutôt que la transformation directe des données.

Les avantages de ce type de réseau sont multiples. Ils permettent

- **Meilleure propagation du gradient** : les connexions résiduelles réduisent le problème du gradient qui disparaît dans les réseaux très profonds.
- **Permet des réseaux très profonds** (jusqu'à plusieurs centaines de couches) tout en restant entraînaibles.
- **Meilleure précision** sur des tâches de classification, détection et segmentation.

Inconvénients :

- **Plus coûteux en calcul** à cause de leur profondeur.
- **Complexité architecturale** plus élevée que les CNN classiques.
- **Risque de surajustement** sur petits jeux de données sans régularisation.

```
def _conv_block(self, inputs, filters, kernel, activation=None):

    x = Conv2D(filters, kernel, padding='same',
               kernel_regularizer=regularizers.l2(0.0001),
               use_bias=False, kernel_initializer='he_normal')(inputs)
    x = BatchNormalization()(x)
    if activation is not None:
        x = Activation(activation)(x)
    return x

def get_iteration_block(self, inputs, filters, kernel, activation):
    x = self._conv_block(inputs, filters, kernel, activation=activation)
    x = self._conv_block(x, filters, kernel, activation=activation)
    x = Add()([inputs, x])
    return x
```

Le bloc **get_iteration_block** représente l'implémentation du bloc résiduel fondamental de l'architecture **ResNet**, basé sur le principe révolutionnaire introduit par He et al. qui a permis l'entraînement efficace de réseaux très profonds.

Ce bloc suit une structure séquentielle en trois étapes distinctes. La première étape applique une convolution complète via la méthode **_conv_block**, transformant le tenseur d'entrée tout en préservant ses dimensions spatiales grâce au padding approprié. Cette convolution inclut automatiquement la normalisation par batch et l'activation spécifiée, créant une première transformation non-linéaire des features d'entrée.

La deuxième étape répète exactement la même opération, appliquant une seconde convolution avec les mêmes paramètres que la première. Cette double convolution constitue le cœur de l'apprentissage du bloc, permettant l'apprentissage de transformations complexes à travers deux couches non-linéaires successives. L'utilisation de kernels identiques dans les deux convolutions maintient la cohérence dimensionnelle et facilite l'optimisation.

L'innovation de ce type d'architecture réside dans la troisième étape : l'addition directe entre le tenseur d'entrée original et le résultat des deux convolutions. Cette connexion résiduelle, également appelée "**skip connection**", permet au réseau d'apprendre des fonctions résiduelles plutôt que des transformations absolues. Mathématiquement, au lieu d'apprendre une fonction $H(x)$, le bloc apprend $F(x) = H(x) - x$, où la sortie finale devient $F(x) + x$.

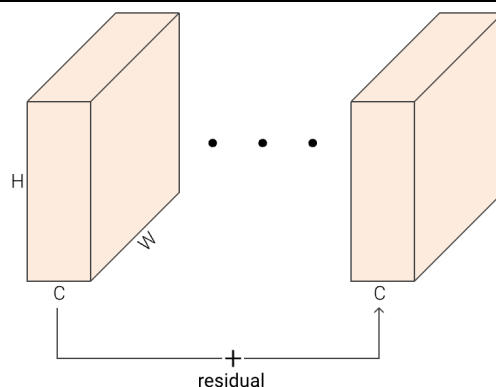


Fig : Residual Block

Cette approche résout élégamment le problème de dégradation des gradients qui affecte les réseaux profonds traditionnels. La connexion directe garantit qu'au minimum, l'information peut traverser le bloc sans altération, même si les **couches convolutionnelles** ne contribuent pas positivement. Cela facilite grandement la propagation des gradients lors de la rétropropagation, permettant l'entraînement stable de réseaux comportant des dizaines ou centaines de couches.

La connexion résiduelle encourage également l'apprentissage incrémental, où chaque bloc affine progressivement les représentations plutôt que de les transformer radicalement. Cette propriété améliore la stabilité d'entraînement et permet une convergence plus rapide vers des solutions optimales, expliquant le succès remarquable des architectures ResNet dans de nombreux domaines d'application.

3.4.2 Modèle MobileNet adapté au Go

3.4.2.1 Les concepts

Révolution architecturale : les convolutions séparables

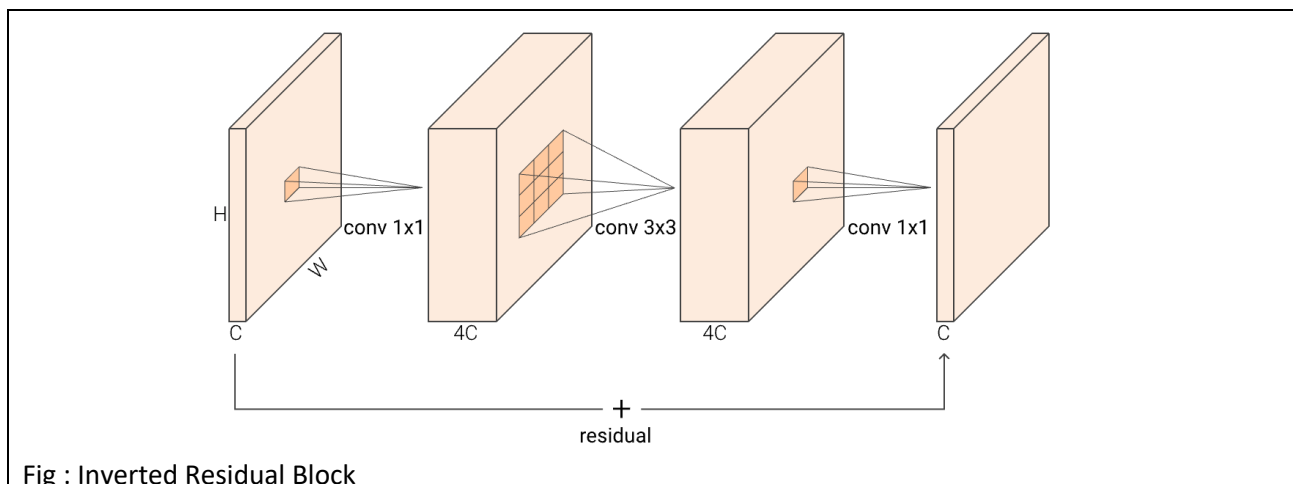
L'architecture MobileNet représente un changement paradigmatique dans la conception des réseaux de neurones convolutionnels, introduisant une innovation fondamentale qui révolutionne l'efficacité computationnelle sans sacrifier les performances. Cette innovation repose sur la **décomposition des convolutions standard** en deux opérations distinctes et séquentielles : les convolutions séparables en profondeur (depthwise) et les convolutions pointwise.

Contrairement aux convolutions traditionnelles qui mélangent simultanément l'information spatiale et les canaux, MobileNet sépare ces deux processus. La convolution depthwise applique un filtre unique à chaque canal d'entrée indépendamment, capturant les corrélations spatiales sans mélange inter-canaux. La convolution pointwise suivante utilise des kernels 1×1 pour combiner l'information entre canaux sans dimension spatiale. Cette factorisation réduit drastiquement la complexité computationnelle : une convolution standard 3×3 avec C canaux d'entrée et N canaux de sortie nécessite $9 \times C \times N$ multiplications par position spatiale, tandis que la décomposition MobileNet n'en requiert que $9 \times C + C \times N$.

Architecture bottleneck inversée

MobileNet V2 introduit le concept révolutionnaire de **bottleneck inversé**, qui contraste radicalement avec l'approche ResNet traditionnelle. MobileNet adopte la stratégie : expansion $\rightarrow$ traitement $\rightarrow$ compression. Cette stratégie s'appuie sur l'observation que les convolutions depthwise opèrent plus efficacement sur des représentations de haute dimension.

Le bloc bottleneck inversé débute par une expansion linéaire qui multiplie le nombre de canaux par un facteur donné (typiquement 4 ou 6), créant un espace représentationnel enrichi. La convolution depthwise opère ensuite sur cet espace étendu, permettant l'extraction de features plus nuancées. Enfin, une projection linéaire via convolution 1×1 compresse les informations vers l'espace de sortie, souvent plus compact que l'entrée. Cette architecture préserve l'information critique tout en maintenant une efficacité computationnelle optimale.



Mécanisme Squeeze-and-Excitation

L'intégration du mécanisme **Squeeze-and-Excitation (SE)** ajoute une dimension d'attention sophistiquée aux réseaux MobileNet. Ce mécanisme implémente un système d'attention adaptatif sur les canaux, permettant au réseau de recalibrer dynamiquement l'importance relative des différentes features. Le processus squeeze effectue un pooling global pour résumer chaque canal en un descripteur scalaire, tandis que l'opération excitation utilise un petit réseau fully-connected pour prédire les poids d'importance de chaque canal.

Cette attention sur les canaux s'avère particulièrement pertinente pour le Go, où différents plans d'encodage (libertés, échelles, historique) possèdent des importances variables selon le contexte positionnel. Le mécanisme SE permet au réseau d'adapter automatiquement sa focalisation, privilégiant par exemple les informations tactiques en situation de combat ou les patterns territoriaux en phase d'ouverture.

3.4.2.2 Le Code

```
def _depthwise_conv_block(self, inputs, kernel, activation):
    x = DepthwiseConv2D(kernel, depth_multiplier=1, padding='same', use_bias=False,
                        kernel_regularizer=regularizers.l2(0.0001),
                        kernel_initializer='he_normal')(inputs)
    x = BatchNormalization()(x)
    x = Activation(activation)(x)
    return x

def _squeeze_block(self, input, ratio=16):
    filters = input.shape[-1]
    se = GlobalAveragePooling2D()(input)
    se = Reshape((1, 1, filters))(se)
    se = Dense(filters // ratio, activation='relu', use_bias=False)(se)
    se = Dense(filters, activation='sigmoid', use_bias=False)(se)
    x = Multiply()([input, se])
    return x
```

```

def _nested_bottleneck_block(self, inputs, filters, kernel, factor, squeeze,
activation):
    # Expansion basée sur les canaux d'entrée
    input_filters = inputs.shape[-1]
    expansion = int(input_filters * factor)

    # Expansion 1x1
    x = self._conv_block(inputs, expansion, (1, 1), activation)

    # Depthwise convolution
    x = self._depthwise_conv_block(x, kernel, activation)

    # Squeeze & Excitation optionnel
    if squeeze:
        x = self._squeeze_block(x)

    # Projection 1x1 vers les canaux de sortie
    x = self._conv_block(x, filters, (1, 1), activation)

    # ✅ Connexion résiduelle seulement si dimensions compatibles
    if input_filters == filters:
        x = Add()(x, inputs)

```

`_depthwise_conv_block` : Convolution séparable en profondeur

Cette méthode implémente la convolution depthwise caractéristique de MobileNet. Contrairement aux convolutions classiques, elle applique un filtre unique à chaque canal d'entrée indépendamment, sans mélange inter-canaux. Le paramètre `depth_multiplier=1` maintient le même nombre de canaux en sortie. Cette opération capture les corrélations spatiales locales tout en préservant l'efficacité computationnelle. La normalisation par batch et l'activation complètent le bloc selon le pattern standard.

`_squeeze_block` : Mécanisme d'attention sur les canaux

Cette méthode implémente le mécanisme Squeeze-and-Excitation qui ajoute un système d'attention adaptatif. Elle commence par un pooling global qui résume chaque canal en un scalaire, puis applique deux couches denses pour créer des poids d'attention : la première réduit la dimensionnalité (division par ratio), la seconde la restaure avec activation sigmoïde pour produire des coefficients entre 0 et 1. Ces poids sont finalement multipliés élément par élément avec l'entrée originale, permettant au réseau de privilégier automatiquement les canaux les plus informatifs.

`_nested_bottleneck_block` : Bloc bottleneck inversé complet

Cette méthode orchestre la séquence complète du bottleneck inversé MobileNet. Elle débute par une expansion linéaire via convolution 1x1 qui multiplie les canaux par le facteur spécifié, créant un espace représentationnel enrichi. Suit la convolution depthwise sur cet espace étendu, puis optionnellement le mécanisme Squeeze-and-Excitation pour l'attention. Une projection finale via convolution 1x1 compresse vers les canaux de sortie. La connexion résiduelle s'ajoute uniquement si les dimensions d'entrée et de sortie sont compatibles, préservant l'information tout en permettant l'apprentissage de transformations complexes.

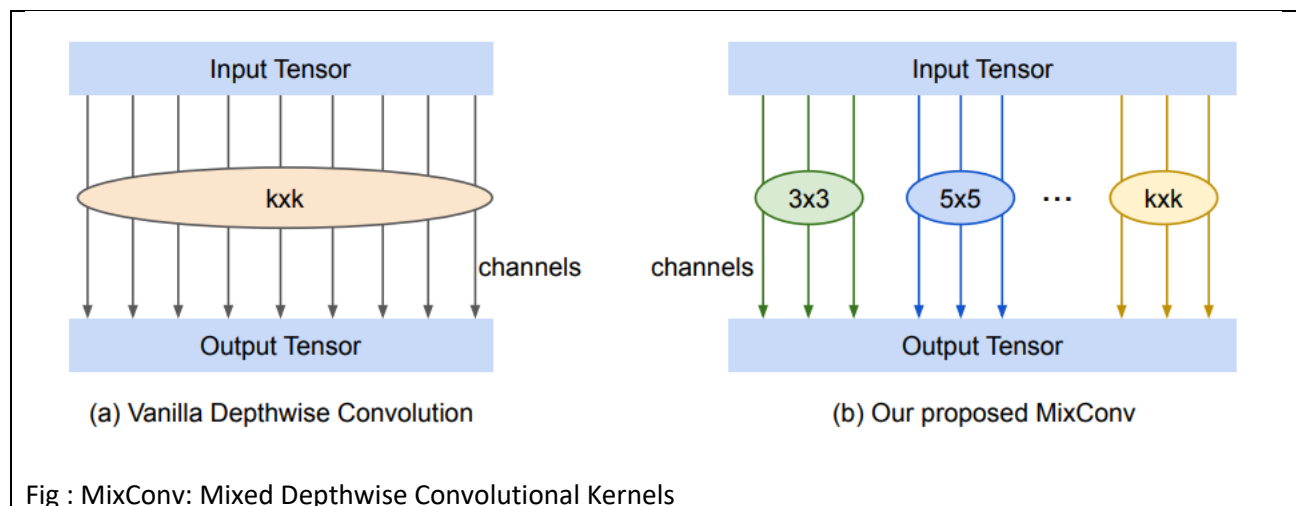
3.4.3 Modèle MixNet adapté au Go

3.4.3.1 Concept innovant des convolutions mixtes (MixConv)

L'architecture MixNet introduit une innovation fondamentale dans le domaine des convolutions séparables via le concept de **MixConv** (Mixed Convolutions). Cette approche révolutionnaire répond à une limitation intrinsèque des convolutions depthwise traditionnelles : la contrainte d'utiliser une taille de kernel unique pour tous les canaux. Tan et al. ont démontré que cette uniformité constitue un goulot d'étranglement expressif, empêchant l'adaptation optimale des récepteurs selon la nature des features à extraire.

Le principe fondamental de MixConv repose sur la **diversification des champs récepteurs** au sein d'une même couche convolutionnelle. Plutôt que d'appliquer uniformément des kernels 3×3 à tous les canaux, MixConv partitionne les canaux en groupes distincts et leur associe des tailles de kernels différentes. Cette approche permet de capturer simultanément des patterns à multiples échelles spatiales : les kernels de petite taille (3×3) excellent dans la détection de motifs locaux et de contours fins, tandis que les kernels plus larges (5×5, 7×7) capturent des structures spatiales étendues et des corrélations à long terme.

Dans notre implémentation adaptée au Go, nous combinons spécifiquement des kernels 3×3 et 5×5 selon une répartition équilibrée. Cette combinaison s'avère particulièrement pertinente pour le jeu de Go, où coexistent des patterns tactiques locaux (captures, connexions immédiates) et des formations stratégiques étendues (moyo, influence territoriale). La division des canaux permet d'allouer une moitié aux analyses locales et l'autre moitié aux évaluations globales, créant une représentation dual-scale intrinsèque.



3.4.3.2 Le Code

```
# --- MixDepthwise Block ---
def _mix_depthwise_conv(self, inputs, activation):
    # Le nombre de canaux est déterminé par l'input
    input_channels = inputs.shape[-1]
    channels_3x3 = input_channels // 2
    channels_5x5 = input_channels - channels_3x3
```

```

x1 = DepthwiseConv2D((3, 3),
                      padding='same', depth_multiplier=1, use_bias=False,
                      depthwise_regularizer=regularizers.l2(0.0001))(inputs)
x2 = DepthwiseConv2D((5, 5),
                      padding='same', depth_multiplier=1, use_bias=False,
                      depthwise_regularizer=regularizers.l2(0.0001))(inputs)

# Sélection des canaux appropriés
x = Concatenate()([x1[:, :, :, :channels_3x3], x2[:, :, :, :channels_5x5]])
x = BatchNormalization()(x)
x = Activation(activation)(x)
return x

def _bottleneck_mix_block(self, inputs, filters, factor, squeeze, activation):
    input_filters = inputs.shape[-1]
    expansion = int(input_filters * factor) # ✅ Basé sur input_filters

    # Expansion 1x1
    x = self._conv_block(inputs, expansion, (1, 1), activation)

    # MixDepthwise convolution (ne prend que inputs et activation)
    x = self._mix_depthwise_conv(x, activation) # ✅ Plus de paramètre filters

    # Squeeze & Excitation optionnel
    if squeeze:
        x = self._squeeze_block(x)

    # Projection 1x1 vers les canaux de sortie
    x = self._conv_block(x, filters, (1, 1), activation)

    # Connexion résiduelle seulement si dimensions compatibles
    if input_filters == filters:
        x = Add()([x, inputs])

    return x

def build(self):
    pass

```

Architecture MixDepthwise

Le bloc MixDepthwise représente une amélioration des convolutions depthwise traditionnelles en mélangeant plusieurs tailles de noyaux de convolution pour capturer différents types de motifs à différentes échelles. Cette approche permet de surmonter les limitations des convolutions depthwise classiques qui utilisent une seule taille de noyau.

Le bloc MixDepthwise divise les canaux d'entrée en deux parties distinctes. La première moitié est traitée par une convolution depthwise avec un noyau 3x3, tandis que la seconde moitié utilise un noyau 5x5. Cette partition permet d'exploiter les avantages des petits noyaux pour capturer les détails fins et des noyaux plus grands pour les motifs plus larges.

La fonction `_mix_depthwise_conv` détermine automatiquement le nombre de canaux pour chaque taille de noyau en fonction des dimensions d'entrée. Les canaux sont répartis équitablement, avec la moitié assignée aux convolutions 3x3 et le reste aux convolutions 5x5. Une régularisation L2 est appliquée pour éviter le surapprentissage.

Structure du bloc bottleneck

Le bloc bottleneck MixDepthwise suit l'architecture classique des réseaux mobiles avec une phase d'expansion, une convolution depthwise mixte, et une phase de projection. L'expansion augmente le nombre de canaux selon un facteur multiplicatif basé sur les canaux d'entrée. La convolution MixDepthwise traite ensuite ces canaux étendus avec les deux tailles de noyaux simultanément.

Le mécanisme de Squeeze & Excitation est intégré optionnellement pour ajouter une attention sur les canaux. La phase finale de projection réduit le nombre de canaux vers la dimension de sortie désirée. Une connexion résiduelle est établie lorsque les dimensions d'entrée et de sortie sont compatibles.

4. Procédure d'entraînements

L'entraînement de réseaux de neurones profonds pour le jeu de Go représente un défi technique majeur nécessitant une approche méthodologique rigoureuse. Cette section présente la procédure d'entraînement développée.

4.1 Configuration et Initialisation

La configuration du système utilise un format JSON structuré permettant de définir l'ensemble des hyperparamètres de manière centralisée. Cette approche garantit la reproductibilité des expériences et facilite l'exploration systématique de l'espace des hyperparamètres. Les paramètres incluent la structure du réseau test, les réglages d'optimisation, ainsi que les configurations d'augmentation.

```
CONFIG = {
  'entity': 'Emiasd',
  'project': 'GoProject',
  'experiment_name': 'Test-FinalAttempt3.1',
  'model': {
    'type': 'GoMobilNet',
    'block_num': 7,
    'filters': 32,
    'factor': 4,
    'se': True,
    'drop_out_rate': 0.2,
    'activation': 'swish',
  },
  'train': {
    'batch': 32,
    'initial_lr': 0.0008,
    'alpha_lr': 0.000001,
    'clipnorm_lr': 0.5,
    'first_decay_steps_lr': 0,
```

```

        't_mul_lr': 1.0,
        'm_mul_lr': 1.0,
        'nb_cosinedecay_cycle': 1,
        'policy_weight': 1.0,
        'value_weight': 4.0,
        'epochs': 300,
        'N': 25000,
        'validation_frequency': 20
    },
    'augmentation': {
        'use_augmentation': True,
        'mode': 'all_transforms_per_batch',
        'nb_rotation': 8,
        'transform_probability': 1.0
    }
}

```

4.1.1 Section Expérimentation et Monitoring

```

'entity': 'Emiasd',
'project': 'GoProject',
'experiment_name': 'Test-FinalAttempt3.1',

```

Cette section définit les paramètres de traçabilité. L'entité correspond au compte utilisateur, le projet regroupe les expérimentations liées, et le nom d'expérience permet l'identification unique de chaque run. Cette structuration facilite l'organisation et la comparaison des résultats expérimentaux.

4.1.2 Section Architecture du Modèle

```

'model': {
    'type': 'GoMobilNet',
    'block_num': 7,
    'filters': 32,
    'factor': 4,
    'se': True,
    'drop_out_rate': 0.2,
    'activation': 'swish',
},

```

Les paramètres architecturaux définissent la structure du réseau mobile adapté au Go. Le nombre de blocs (block_num: 7) détermine la profondeur du réseau, inspiré des travaux de Cazenave sur l'équilibre optimal entre profondeur et largeur des réseaux mobiles. Le nombre de filtres (filters: 32) contrôle la largeur de base du réseau, avec un facteur d'expansion (factor: 4) pour les couches internes des blocs inversés résiduels.

L'activation Squeeze & Excitation (se: True) ajoute un mécanisme d'attention sur les canaux, conformément aux améliorations proposées dans "Improving Model and Search for Computer Go". Le taux de dropout (drop_out_rate: 0.2) régularise l'entraînement pour prévenir le surapprentissage. La fonction d'activation

Swish (activation: 'swish') remplace ReLU selon les recommandations de Cazenave et al., démontrant des améliorations significatives pour les applications de Go.

Ces paramètre évolue en fonction des type d'architecture que nous testons pour rester en dessous des 100 000 paramètres imposé par les contraintes du projet.

4.1.3 Section Entraînement et Optimisation

```
'train': {
    'batch':32,
    'initial_lr':0.0008,
    'alpha_lr': 0.000001,
    'clipnorm_lr':0.5,
    'first_decay_steps_lr': 0,
    't_mul_lr': 1.0,
    'm_mul_lr': 1.0,
    'nb_cosinedecay_cycle': 1,
    'policy_weight':1.0,
    'value_weight':4.0,
    'epochs':300,
    'N':25000,
    'validation_frequency':20
},
```

Cette section configure l'optimisation selon la stratégie de **LR Cosine Decay** avec redémarrages. La taille de batch (batch: 32) équilibre efficacité mémoire et stabilité de l'entraînement. Le learning rate initial (initial_lr: 0.0008) et final (alpha_lr: 0.000001) définissent la plage d'exploration de l'optimiseur AdamW.

Le gradient clipping (clipnorm_lr: 0.5) stabilise l'entraînement des réseaux profonds. Les paramètres t_mul_lr et m_mul_lr contrôlent l'évolution des cycles de LR Cosine Decay: t_mul: 1.0 maintient des cycles de longueur constante, tandis que m_mul: 1.0 conserve l'amplitude des oscillations. Un seul cycle (nb_cosinedecay_cycle: 1) est utilisé conformément aux observations de Cazenave et al. sur l'efficacité du LR Cosine Decay sans redémarrages multiples.

La pondération des pertes (policy_weight: 1.0, value_weight: 4.0) reflète l'importance relative des deux tâches d'apprentissage. Cette pondération 1:4 privilégie l'apprentissage de l'évaluation de position par rapport à la prédiction de coups, optimisant ainsi la qualité des estimations de valeur essentielles pour la recherche MCTS.

La configuration d'entraînement définit 300 époques avec 25000 échantillons par époque, soit 7.5 millions d'échantillons au total (ou 60 millions en mode augmentation exhaustive). La validation s'effectue toutes les 20 époques pour surveiller la généralisation sans surcharger le processus d'entraînement.

4.1.3 Section Augmentation de Données

Le système d'augmentation propose trois configurations distinctes adaptées à différents contextes d'utilisation et contraintes computationnelles :

Configuration 1 : Sans Augmentation

```
'augmentation':{  
  'use_augmentation': False  
}
```

Cette configuration désactive complètement l'augmentation de données, utilisant uniquement les données originales du dataset KataGo. Ce mode sert de référence pour évaluer l'impact des stratégies d'augmentation et convient aux situations où les ressources computationnelles sont limitées ou pour des expérimentations de validation rapide.

Configuration 2 : Mode Aléatoire par Batch

```
'augmentation':{  
  'use_augmentation': True,  
  'mode': 'random_per_batch',  
  'nb_rotation': 8,  
  'transform_probability': 1.0  
}
```

Le mode aléatoire applique une transformation unique sélectionnée aléatoirement à l'ensemble du batch courant. Cette approche offre un excellent compromis entre diversité des données et efficacité computationnelle.

Configuration 3 : Mode Exhaustif par Batch

```
'augmentation':{  
  'use_augmentation': True,  
  'mode': 'all_transforms_per_batch',  
  'nb_rotation': 8,  
  'transform_probability': 1.0  
}
```

Le mode exhaustif applique séquentiellement l'ensemble des 8 transformations géométriques possibles à chaque batch original, maximisant l'exploitation des symétries du plateau de Go.

4.2 Optimiseur AdamW

Notre procédure d'entraînement utilise une stratégie adaptative de sélection d'optimiseur basée sur la configuration du learning rate, privilégiant AdamW pour sa gestion supérieure de la régularisation.

L'optimiseur AdamW, proposé par Loshchilov et Hutter, corrige une limitation fondamentale d'Adam classique dans la gestion du weight decay. La mise à jour des paramètres s'effectue selon :

$$\begin{aligned}
m_t &= \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \nabla \theta_t \\
v_t &= \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot \nabla \theta_t^2 \\
\theta_{t+1} &= \theta_t - \eta_t \cdot (\hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon) + \lambda \cdot \theta_t)
\end{aligned}$$

où $\lambda \cdot \theta_t$ représente le terme de weight decay découplé, appliqué directement aux paramètres plutôt qu'aux gradients.

L'optimiseur Adam classique intègre la régularisation L2 (weight decay) directement dans le calcul des gradients, créant un couplage problématique entre la régularisation et l'adaptation du learning rate. Dans cette formulation traditionnelle, le terme de régularisation est traité comme une composante supplémentaire du gradient, ce qui signifie que son influence effective varie proportionnellement aux ajustements adaptatifs du learning rate. Cette approche pose des défis particuliers lors de l'utilisation de schedules de learning rate agressifs comme le **Cosine Anneling**, où les variations importantes du taux d'apprentissage peuvent compromettre l'efficacité de la régularisation.

AdamW (Adam with Decoupled Weight Decay) résout cette limitation fondamentale en découplant complètement le weight decay de l'optimisation adaptative des gradients. Dans cette formulation améliorée, la régularisation L2 est appliquée directement aux paramètres après la mise à jour adaptative, plutôt que d'être intégrée dans le calcul des gradients. Cette séparation permet au coefficient de weight decay λ d'agir de manière indépendante du learning rate, préservant ainsi l'intention originale de la régularisation indépendamment des variations du taux d'apprentissage.

Cette distinction devient particulièrement critique dans le contexte de l'entraînement de réseaux de neurones pour le Go, où l'utilisation du **Cosine Anneling** impose des variations cycliques importantes du learning rate. Avec Adam classique, ces fluctuations affectent indirectement l'intensité de la régularisation, pouvant conduire à des phénomènes de sous-régularisation lors des phases de learning rate élevé et de sur-régularisation durant les phases de learning rate faible. AdamW maintient une régularisation constante et prévisible tout au long de ces cycles, garantissant une convergence plus stable et une meilleure généralisation du modèle final.

4.3 Optimisation et Scheduling du Learning Rate

4.3.1 Cosine Decay avec Redémarrages

Notre procédure d'entraînement emploie une stratégie d'optimisation avancée basée sur le **Cosine Annealing with Warm Restarts**, directement inspirée des travaux récents de Cazenave, Sentuc et Videau sur l'amélioration de l'apprentissage supervisé pour le Go. Dans leur article "Cosine Annealing, Mixnet and Swish

Activation for Computer Go", les auteurs démontrent l'efficacité supérieure du **Cosine Anneling** par rapport aux méthodes traditionnelles de division du learning rate.

Le principe consiste à faire varier cycliquement le learning rate selon une fonction cosinus, avec des redémarrages périodiques à la valeur initiale. Cette stratégie permet d'explorer plus efficacement l'espace des paramètres et peut aider le modèle à échapper aux minima locaux. La formule utilisée, telle que présentée par Loshchilov et Hutter dans leurs travaux sur SGDR, est :

$$\eta_t = \eta_{\min} + (1/2)(\eta_{\max} - \eta_{\min})(1 + \cos(\pi T_{\text{cur}}/T_i))$$

où η_t représente le learning rate à l'instant t , T_{cur} le nombre de pas depuis le dernier redémarrage, et T_i la durée du cycle courant.

Les résultats expérimentaux de Cazenave et al. Montrent que « cosine annealing ends up with better accuracy and MSE. Moreover, the learning curve for cosine annealing is smoother, for instance there are no bumps on the learning curve because of learning rate changes », confirmant l'intérêt de cette approche pour l'entraînement de réseaux de Go.

4.3.2 Adaptation Dynamique aux Modes d'Augmentation

Un aspect innovant de notre approche réside dans l'adaptation automatique du scheduler aux différents modes d'augmentation. En mode exhaustif, le nombre effectif de batches par époque est multiplié par le nombre de transformations appliquées, nécessitant un ajustement correspondant des paramètres du scheduler pour maintenir la cohérence des cycles d'optimisation.

Cette adaptation garantit que la durée effective des cycles reste proportionnelle au volume de données traitées, indépendamment du mode d'augmentation choisi. Cette cohérence est essentielle pour permettre des comparaisons équitables entre les différentes configurations d'entraînement.

4.4 Fonction d'Activation Swish

$$\text{Swish}(x) = x \cdot \text{sigmoid}(x) = x \cdot (1 / (1 + e^{(-x)}))$$

Cette fonction présente des propriétés remarquables qui expliquent son efficacité supérieure à ReLU dans les architectures de réseaux mobiles. Contrairement à ReLU qui présente une coupure nette à zéro, Swish offre une transition lisse avec un gradient non-nul pour les valeurs négatives, permettant un meilleur flux de gradients lors de la rétropropagation.

Comme le précise explicitement Cazenave et al., "We have tested the Swish activation instead of the Rectified Linear Unit activation (ReLU) **in the inverted residual blocks**". Cette spécification est cruciale car l'activation

Swish n'est pas appliquée uniformément dans toute l'architecture, mais uniquement dans les blocs résiduels inversés (inverted residual blocks).

Comme le précise explicitement Cazenave et al., "We have tested the Swish activation instead of the Rectified Linear Unit activation (ReLU) **in the inverted residual blocks**". Cette spécification est cruciale car l'activation Swish n'est pas appliquée uniformément dans toute l'architecture, mais uniquement dans les blocs résiduels inversés (inverted residual blocks).

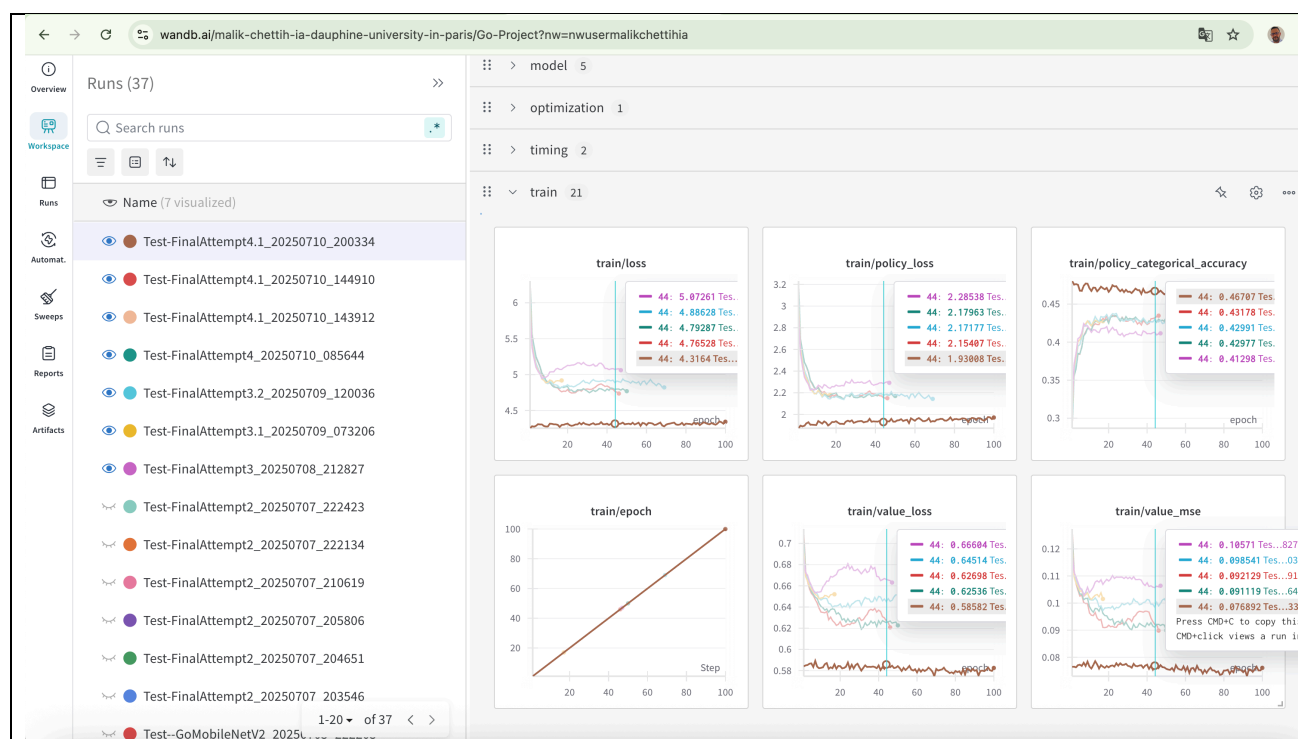


5 Monitoring

5.1 Architecture de Monitoring Intégrée

L'une des contributions majeures de notre procédure d'entraînement réside dans l'implémentation d'un système de monitoring complet et automatisé via la classe `TrainingMonitor`. Cette architecture de surveillance permet un suivi en temps réel de l'entraînement, la détection précoce d'anomalies, et la conservation d'un historique détaillé pour l'analyse comparative des expérimentations.

Le système de monitoring s'intègre nativement avec Weights & Biases (WandB), plateforme de référence pour le suivi d'expérimentations en apprentissage automatique. Cette intégration offre plusieurs avantages cruciaux : la centralisation des métriques de multiples runs d'entraînement, la visualisation interactive des courbes de convergence, la comparaison automatisée des hyperparamètres, et la reproductibilité complète des expérimentations grâce à la sauvegarde systématique des configurations.



5.2 Gestion Robuste des Métriques Hétérogènes

Un défi technique majeur dans le monitoring d'entraînements complexes réside dans la gestion de métriques aux formats hétérogènes. Notre implémentation résout cette problématique via la méthode `_safe_convert_to_float`, qui effectue une conversion intelligente des données vers des formats compatibles avec WandB. Cette fonction gère automatiquement les tensors TensorFlow, les arrays NumPy, les listes de valeurs, et les types scalaires, garantissant ainsi une logging robuste indépendamment du format source des métriques.

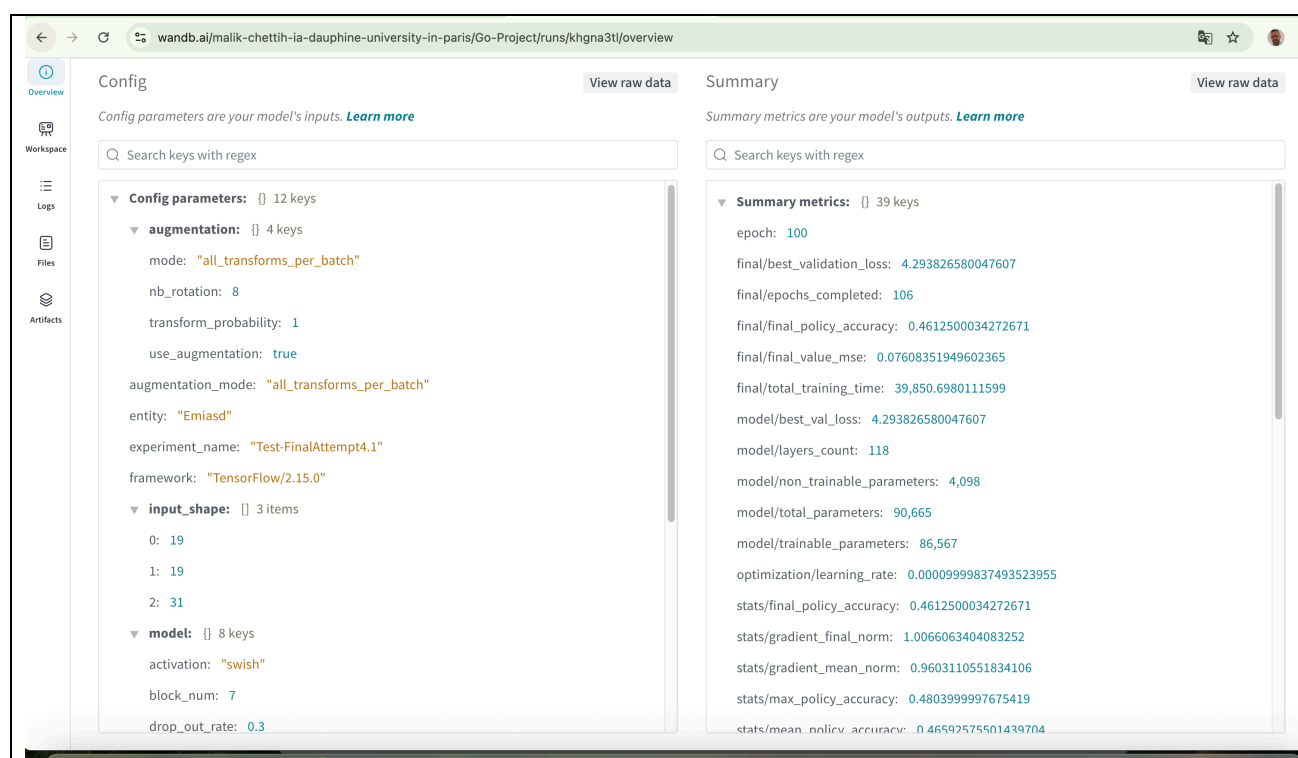
La stratégie de conversion adoptée privilégie la robustesse à la performance, filtrant automatiquement les valeurs non-numériques tout en préservant l'intégrité de l'entraînement. Cette approche défensive évite les interruptions d'entraînement dues à des incompatibilités de format, problème fréquent lors de l'intégration de systèmes de monitoring externes avec des frameworks d'apprentissage automatique.

5.3 Initialisation et Configuration WandB

La méthode `initialize_wandb` orchestre la connexion avec la plateforme **Weights & Biases** en fusionnant les configurations par défaut avec les paramètres spécifiques à l'expérimentation. La configuration par défaut établit un contexte standardisé incluant les dimensions d'entrée (19×19×31 pour le Go), le nombre de classes de sortie (361 positions), et les informations de framework (version TensorFlow).

La stratégie de tagging automatique facilite l'organisation et la recherche des expérimentations dans l'interface WandB. Les tags par défaut ("go", "mobilenet", "neural-network") peuvent être complétés par des tags spécifiques à l'expérimentation, permettant une catégorisation fine des runs selon différents critères (architecture, dataset, objectif expérimental).

La gestion des notes d'expérimentation et la génération d'URLs uniques facilitent la documentation et le partage des résultats. L'option `reinit=True` permet la réutilisation de la même instance de monitoring pour plusieurs runs successifs, optimisant l'utilisation des ressources dans les contextes d'expérimentation intensive.



5.4 Logging des Informations Architecturales

La méthode `log_model_info` extrait et enregistre automatiquement les caractéristiques structurales du modèle entraîné. Cette fonctionnalité calcule le nombre total de paramètres, distingue les paramètres entraînaux des paramètres fixes, et compte les couches constitutives de l'architecture.

Ces métadonnées architecturales s'avèrent essentielles pour l'analyse comparative des performances selon la complexité du modèle. Elles permettent également de détecter automatiquement les incohérences dans la définition d'architectures et facilitent l'estimation des besoins en ressources computationnelles.

The screenshot displays the WandB overview page for a specific run. The left sidebar contains navigation links: Overview, Workspace, Logs, Files, and Artifacts. The main content area is divided into two panels. The left panel shows the 'model' configuration with 8 keys: activation ('swish'), block_num (7), drop_out_rate (0.3), factor (4), filters (32), name ('best_model_epoch240_val2.5550_aug8x.h5'), se (true), type ('GoMobilNet'), nb_rotation (8), num_classes (361), project ('GoProject'), train (13 keys), and transform_probability (1). The right panel shows training statistics, including final/total_training_time (39,850.6980111599), model/best_val_loss (4.293826580047607), model/layers_count (118), model/non_trainable_parameters (4,098), model/total_parameters (90,665), model/trainable_parameters (86,567), optimization/learning_rate (0.00009999837493523955), stats/final_policy_accuracy (0.4612500034272671), stats/gradient_final_norm (1.0066063404083252), stats/gradient_mean_norm (0.9603110551834106), stats/max_policy_accuracy (0.4803999997675419), stats/mean_policy_accuracy (0.46592575501439704), timing/epoch_duration (395.4799380302429), and timing/total_elapsed (39,849.059427022934). Below these panels, the 'Artifact Outputs' section indicates that the run produced artifacts as outputs, with a link to 'Learn more'. A table below this section lists the artifacts:

Type	Name	Consumer count
model	go_mobilenet_best_epoch_1:v61	0
model	go_mobilenet_best_epoch_40:v32	0

5.5 Gestion Avancée des Métriques d'Époque

La méthode `log_epoch_metrics` implémente une stratégie sophistiquée de traitement des métriques qui organise automatiquement les données selon leur provenance (entraînement, validation, timing, optimisation). Cette organisation facilite l'analyse dans l'interface WandB et améliore la lisibilité des courbes de convergence.

Le préfixage automatique des métriques ("train/", "validation/", "timing/", "optimization/") crée une hiérarchie logique dans l'affichage WandB. La normalisation des noms de métriques (suppression des préfixes "val_" redondants) évite la duplication d'informations et améliore la cohérence de l'affichage.

L'horodatage automatique et le calcul du temps écoulé total fournissent un contexte temporel précieux pour l'analyse des performances. Ces informations permettent d'identifier les phases d'entraînement les plus coûteuses et d'optimiser l'allocation des ressources computationnelles.

Overview

Workspace

Logs

Files

Artifacts

model: {} 8 keys

activation: "swish"

block_num: 7

drop_out_rate: 0.3

factor: 4

filters: 32

name: "best_model_epoch240_val2.5550_aug8x.h5"

se: true

type: "GoMobilNet"

nb_rotation: 8

num_classes: 361

project: "GoProject"

train: {} 13 keys

transform_probability: 1

final/total_training_time: 39,850.6980111599

model/best_val_loss: 4.293826580047607

model/layers_count: 118

model/non_trainable_parameters: 4,098

model/total_parameters: 90,665

model/trainable_parameters: 86,567

optimization/learning_rate: 0.00009999837493523955

stats/final_policy_accuracy: 0.4612500034272671

stats/gradient_final_norm: 1.0066063404083252

stats/gradient_mean_norm: 0.9603110551834106

stats/max_policy_accuracy: 0.4803999997675419

stats/mean_policy_accuracy: 0.46592575501439704

timing/epoch_duration: 395.4799380302429

timing/total_elapsed: 39,849.059427022934

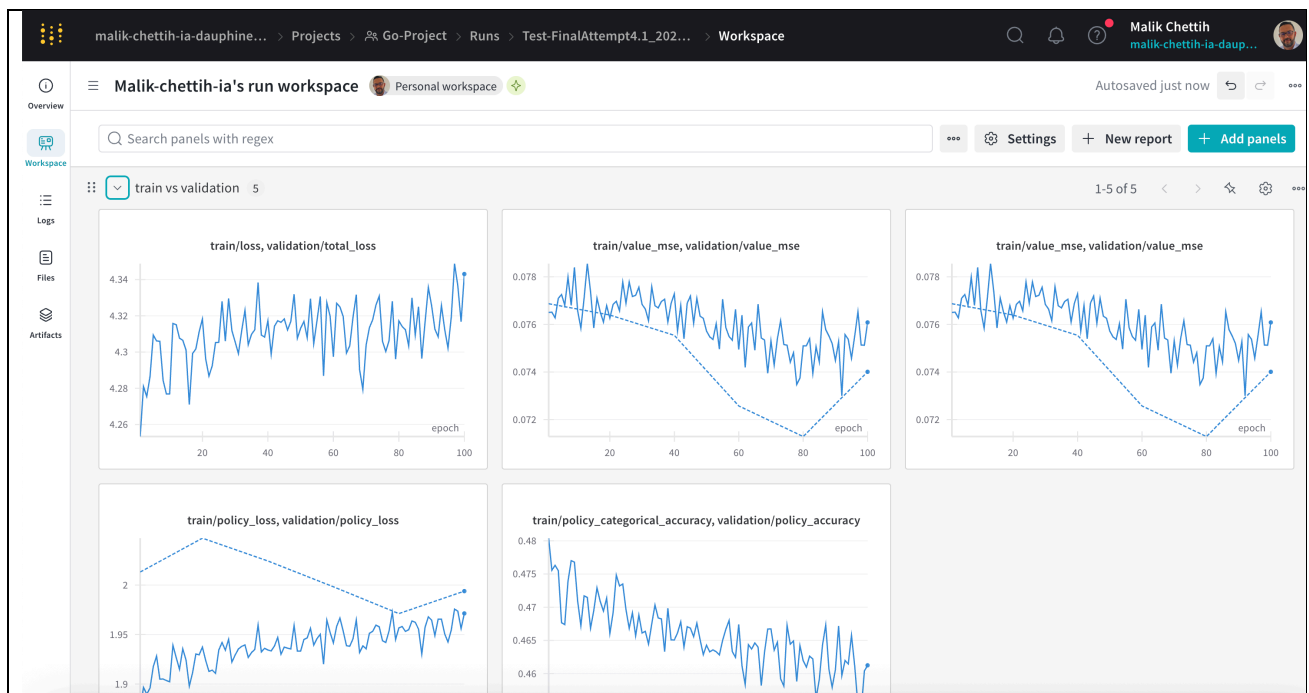
Artifact Outputs

This run produced these artifacts as outputs. [Learn more](#)

Q Search

1-5 of 5

Type	Name	Consumer count
model	go_mobilenet_best_epoch_1:v61	0
model	go_mobilenet_best_epoch_40:v32	0

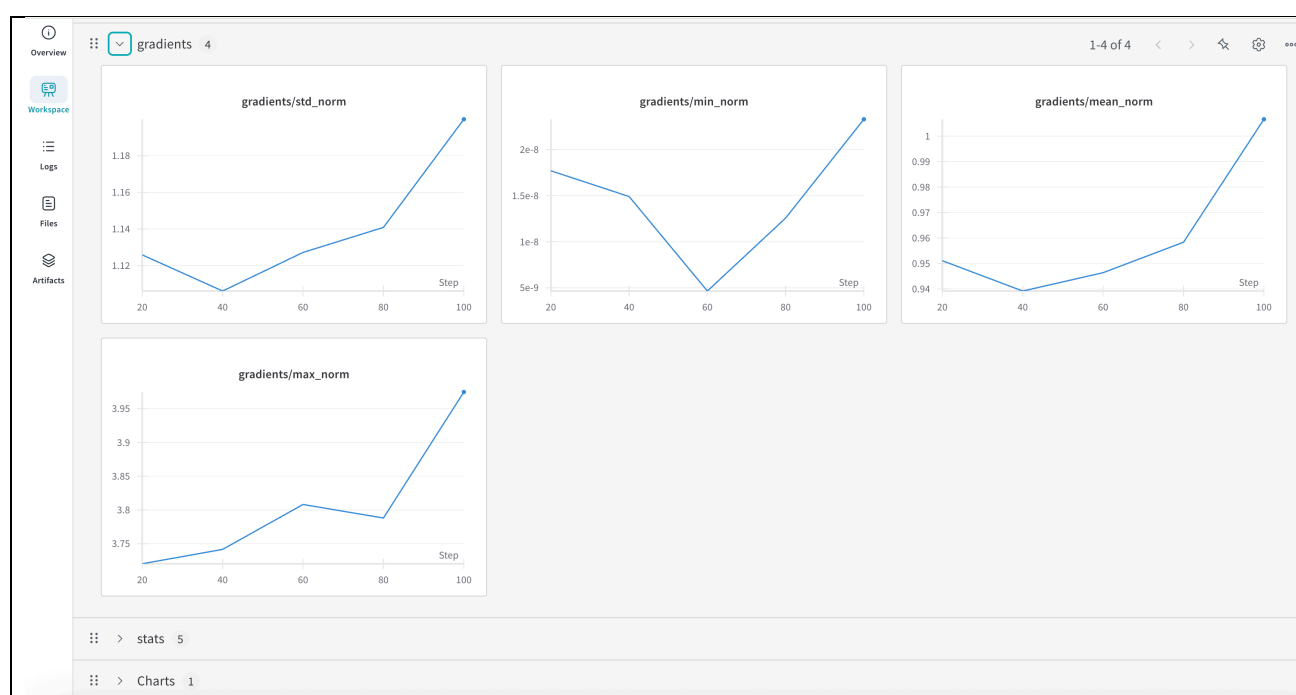


5.6 Diagnostics de Gradients et Détection d'Anomalies

La méthode `log_gradient_metrics` implémente un système de diagnostic avancé qui évalue la santé du processus d'optimisation via l'analyse des gradients. Cette fonctionnalité utilise l'API GradientTape de TensorFlow pour calculer les gradients sur un échantillon représentatif et extraire les normes de gradients pour chaque couche du réseau.

Le calcul des statistiques de gradients (moyenne, maximum, minimum, écart-type) fournit une vue d'ensemble de la distribution des gradients à travers l'architecture. Ces métriques permettent d'identifier les couches problématiques et de diagnostiquer les déséquilibres dans le flux de gradients.

Le système de détection d'anomalies compare automatiquement les normes calculées aux seuils prédéfinis et génère des alertes appropriées. Les seuils de $1e-6$ pour le gradient vanishing et 10.0 pour le gradient explosion ont été calibrés empiriquement sur la base d'expérimentations avec des réseaux mobiles pour le Go.

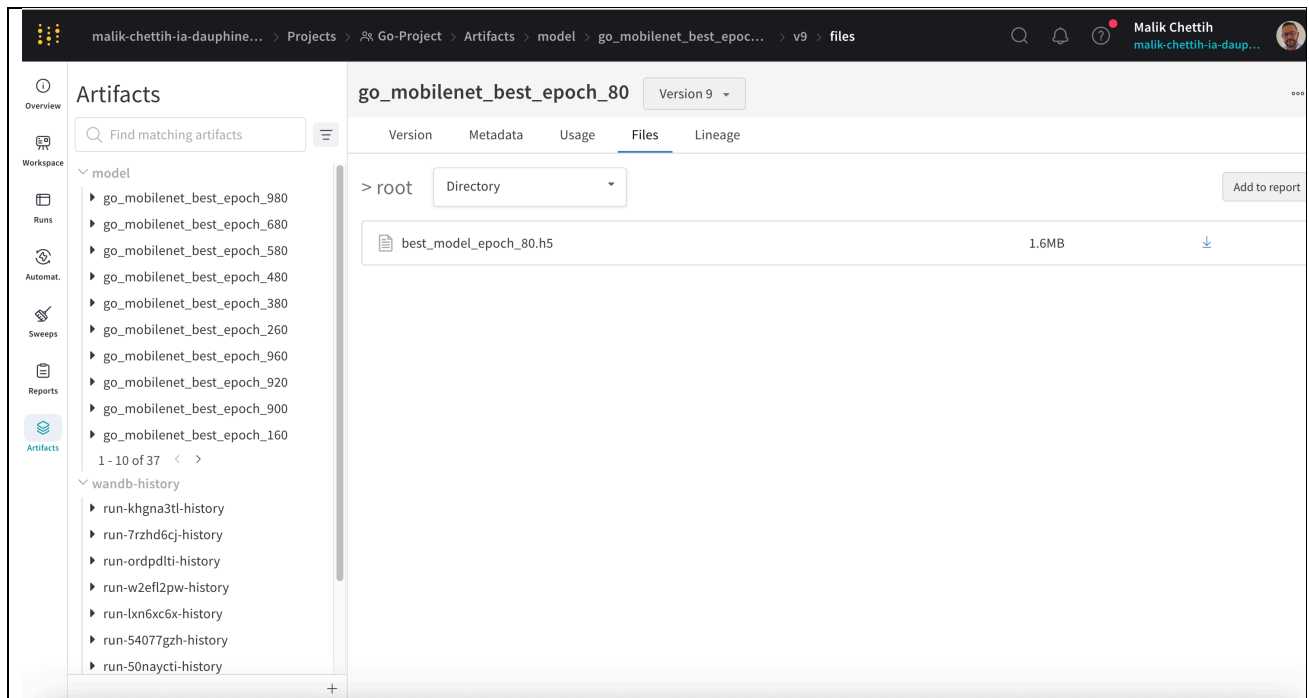


5.7 Sauvegarde Conditionnelle et Gestion des Artifacts

La méthode `log_model_checkpoint` implémente une stratégie de sauvegarde intelligente qui préserve automatiquement les modèles présentant les meilleures performances de validation. Cette approche évite la surcharge de stockage tout en garantissant la conservation des configurations optimales.

Le système de métadonnées associé à chaque checkpoint facilite l'identification et la comparaison des modèles sauvegardés. Les informations stockées incluent l'époque de sauvegarde, la loss de validation, la précision de politique, et l'erreur quadratique moyenne de la valeur.

L'intégration avec le système d'artifacts WandB assure la persistance et la synchronisation des modèles sauvegardés. Cette fonctionnalité facilite le partage de modèles entre membres d'équipe et garantit la reproductibilité des résultats expérimentaux.



5.8 Synthèse Statistique et Analyse de Convergence

La méthode `get_summary_stats` génère automatiquement une synthèse statistique complète de l'entraînement, incluant les performances maximales atteintes, les tendances de convergence, et une évaluation de la santé générale du processus d'apprentissage.

Cette synthèse traite l'historique complet des métriques avec la même robustesse de conversion que les métriques individuelles, garantissant des statistiques fiables même en présence de données partiellement corrompues. L'évaluation de la santé des gradients sur les cinq dernières époques fournit un indicateur de stabilité particulièrement utile pour détecter les problèmes de convergence tardifs.

Les statistiques générées facilitent la comparaison rapide entre différentes configurations d'entraînement et l'identification des patterns de performance récurrents. Ces informations s'avèrent particulièrement précieuses lors de l'exploration systématique d'espaces d'hyperparamètres ou de l'évaluation d'innovations architecturales.

6 Framework de Tests

6.1 Architecture du Framework de Test

L'évaluation rigoureuse de notre procédure d'entraînement nécessite un framework de test structuré et reproductible. Nous avons développé un système modulaire composé de deux éléments principaux : le module utilitaire `go_test_utils.py` qui centralise les fonctionnalités de test et d'analyse, et des scripts d'expérimentation spécialisés comme `go_test00XXX.py` qui définissent les configurations expérimentales spécifiques.

Cette architecture modulaire sépare clairement la logique de test réutilisable des configurations expérimentales, facilitant ainsi la maintenance du code et permettant une exploration systématique de l'espace des hyperparamètres. Le framework supporte nativement la comparaison de multiples architectures et configurations, générant automatiquement des visualisations comparatives et des métriques de performance standardisées.

6.2 Module Utilitaire de Test (`go_test_utils.py`)

6.2.1 Gestion Multi-Architecture

La fonction `get_model` implémente un système de factory pattern pour l'instanciation dynamique de différentes architectures neuronales. Cette approche unifie l'interface de création de modèles tout en supportant la diversité architecturale nécessaire aux études comparatives.

Chaque architecture est configurée avec des paramètres par défaut optimisés empiriquement : GoResNet avec 5 blocs pour un compromis performance/vitesse, GoMobileNet avec 7 blocs selon les recommandations de Cazenave, GoMobileNetV2 avec 9 blocs pour exploiter l'architecture V2 améliorée, et GoMixNet avec 6 blocs optimisés pour les convolutions mixtes.

6.2.2 Orchestration des Expérimentations

La fonction `test_train_model` orchestre l'ensemble du processus expérimental, de l'initialisation du modèle jusqu'à la collecte des résultats finaux. Cette fonction intègre automatiquement le système de monitoring, l'entraînement via GoTrainer, et la collecte structurée des métriques de performance.

L'approche adoptée garantit la cohérence méthodologique entre les différentes expérimentations tout en préservant la flexibilité nécessaire aux études exploratoires. La fonction retourne un tuple structuré contenant le nom d'expérimentation, le modèle entraîné, les métriques de validation, l'historique complet d'entraînement, l'historique de validation, le temps total d'exécution, et l'évolution du learning rate.

6.2.3 Visualisation et Analyse Comparative

Le système de visualisation intégré via `plot_result` génère automatiquement des graphiques comparatifs multi-métriques qui facilitent l'analyse des performances relatives entre différentes configurations. Cette fonction utilise `matplotlib` avec une mise en page sophistiquée (**GridSpec**) pour organiser de manière cohérente les différentes métriques d'intérêt.

Les visualisations incluent l'évolution des pertes totales, des pertes spécialisées (politique et valeur), des métriques de précision, et des erreurs quadratiques moyennes, toutes présentées avec leurs courbes

d'entraînement et de validation correspondante. Cette approche permet d'identifier rapidement les phénomènes de surapprentissage, les problèmes de convergence, et les différences de performance entre architectures.

6.2.4 Analyse des Learning Rates

La fonction **plot_learning_rate** fournit une visualisation dédiée à l'évolution du learning rate durant l'entraînement, particulièrement utile pour valider le comportement des schedulers de type recuit cosinus. Cette analyse permet de vérifier que les cycles d'optimisation se déroulent conformément aux attentes et d'identifier d'éventuelles anomalies dans la planification du learning rate.

6.2.5 Reporting Standardisé

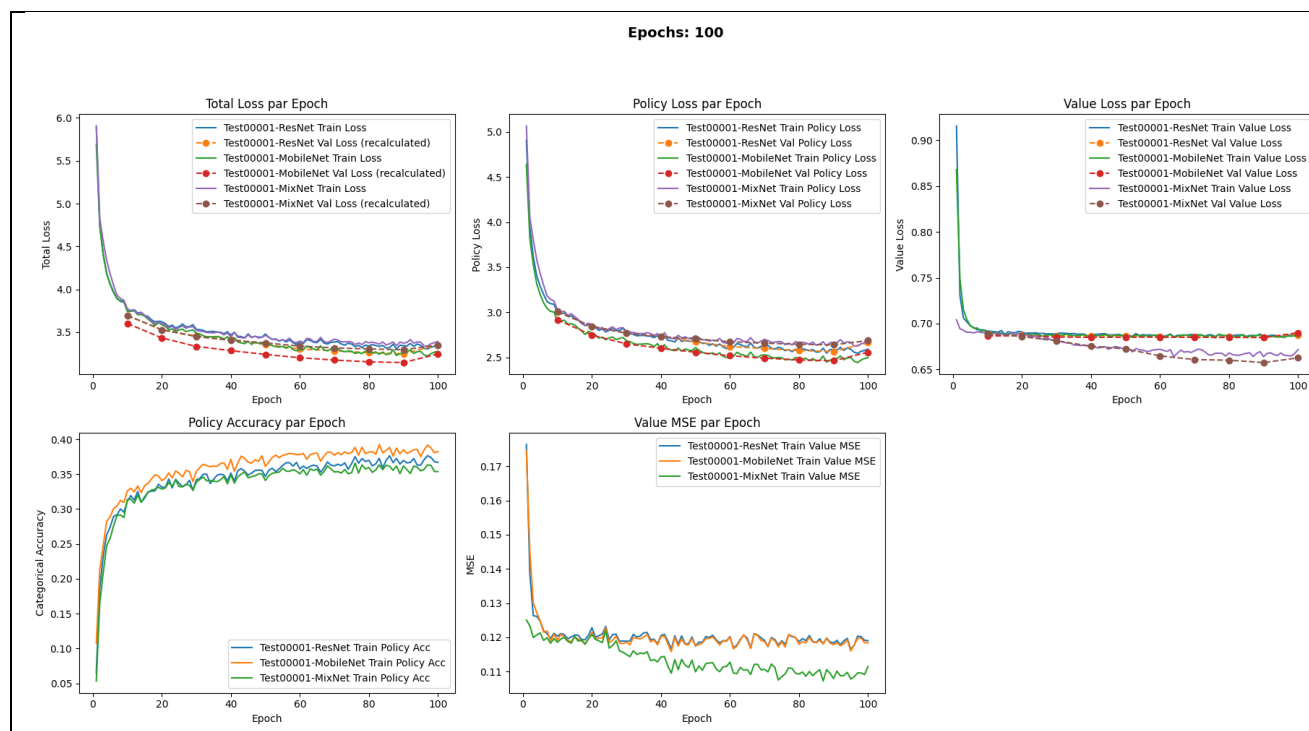
La fonction **print_validation_results** génère des rapports de validation standardisés qui facilitent la comparaison quantitative entre différentes expérimentations. Ces rapports incluent l'ensemble des métriques de validation, les temps d'exécution, et les identificateurs d'expérimentation, permettant une documentation systématique des résultats obtenus.

7 Tests Expérimentaux et Diagnostics

7.1 Démarche expérimental

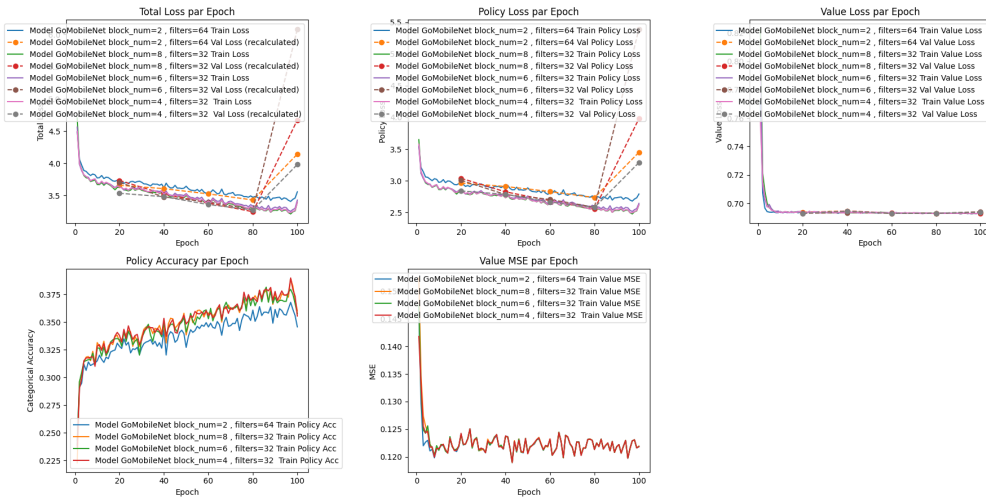
7.2 Tests Expérimentaux

7.2.1 Test différentes architectures de modèles

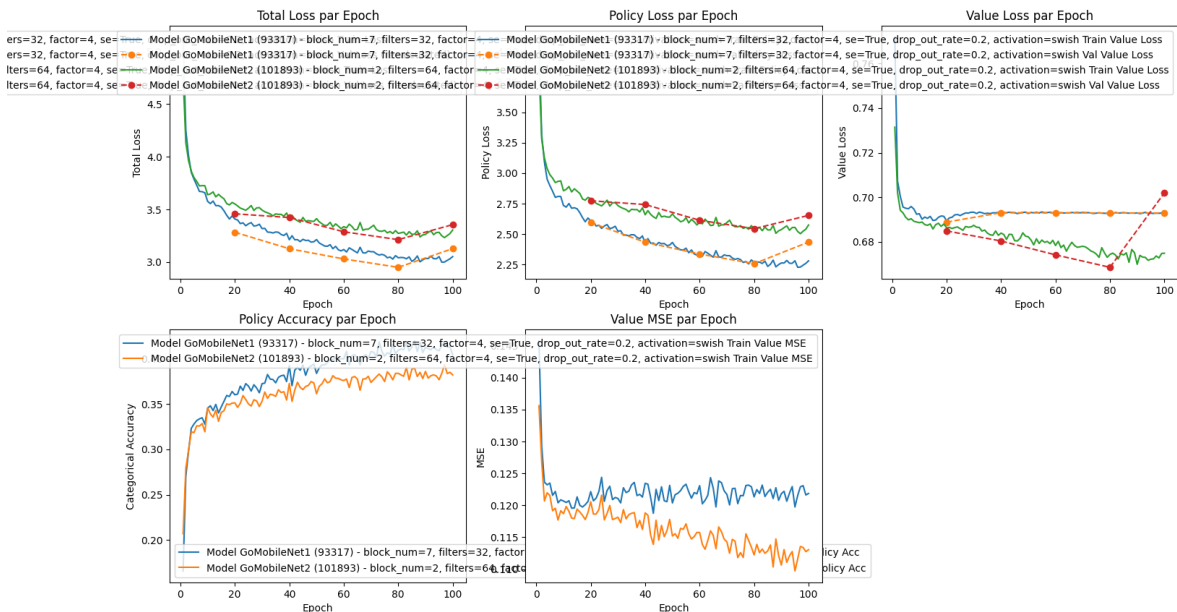


7.2.2 Test de la profondeur du réseau

Epochs: 100

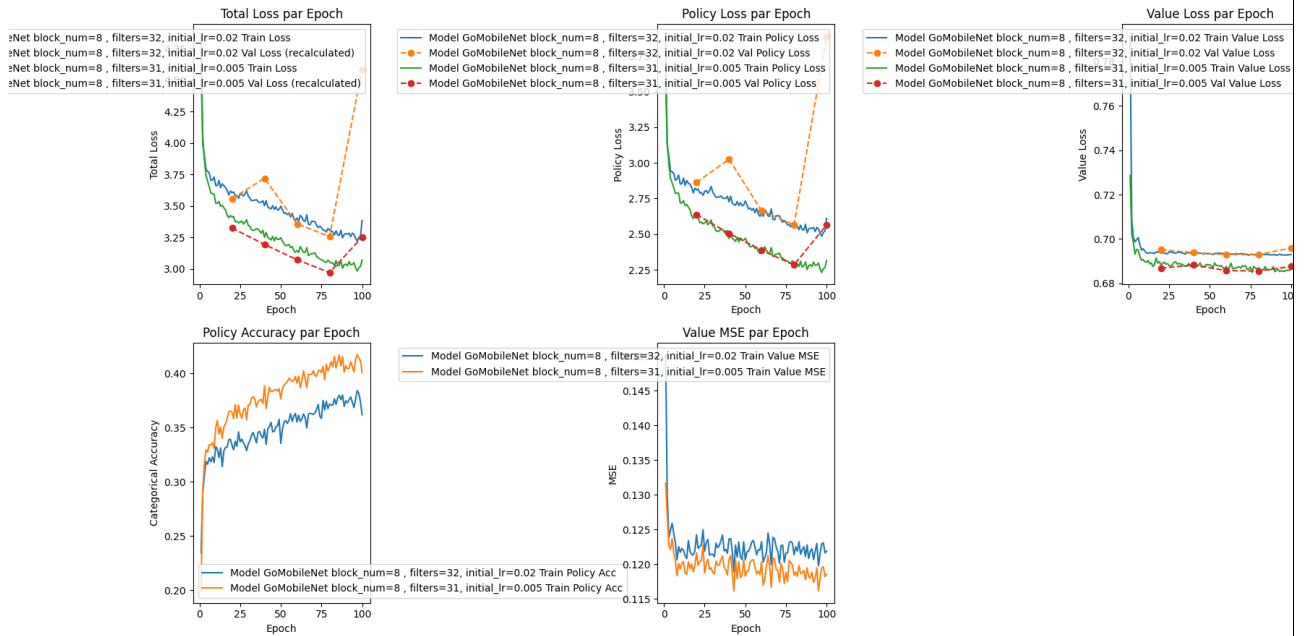


Epochs: 100

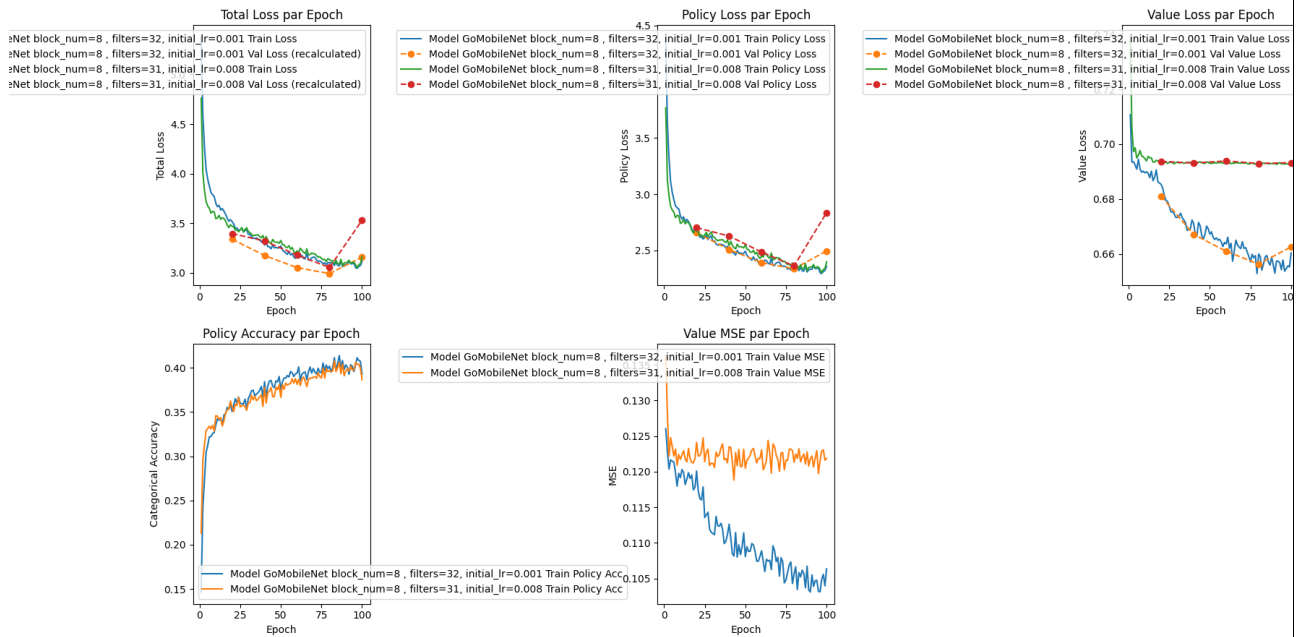


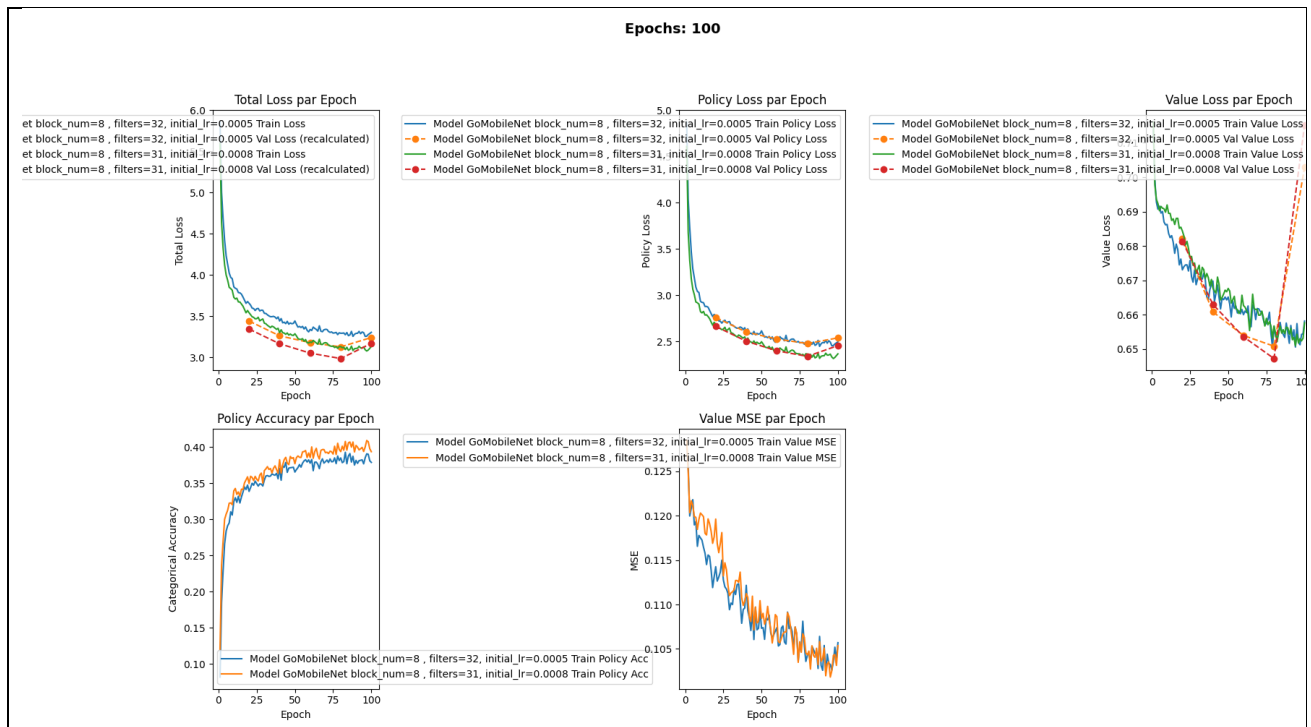
7.2.2 Test Learning rate optimal

Epochs: 100

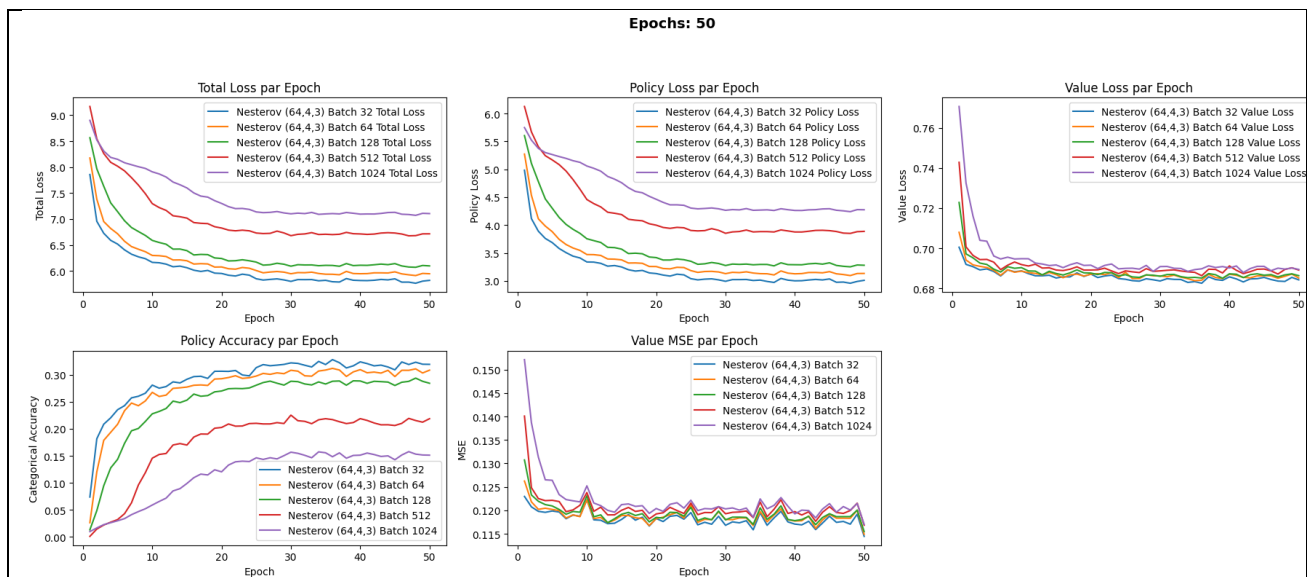


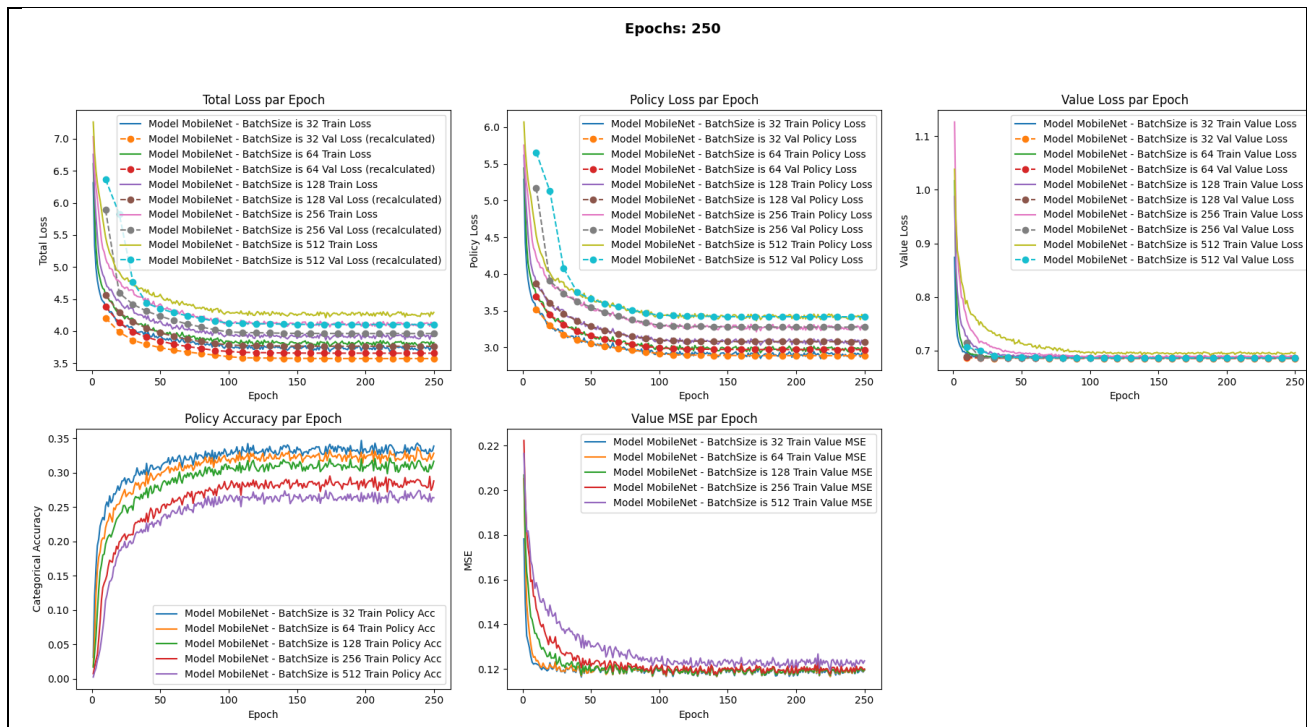
Epochs: 100



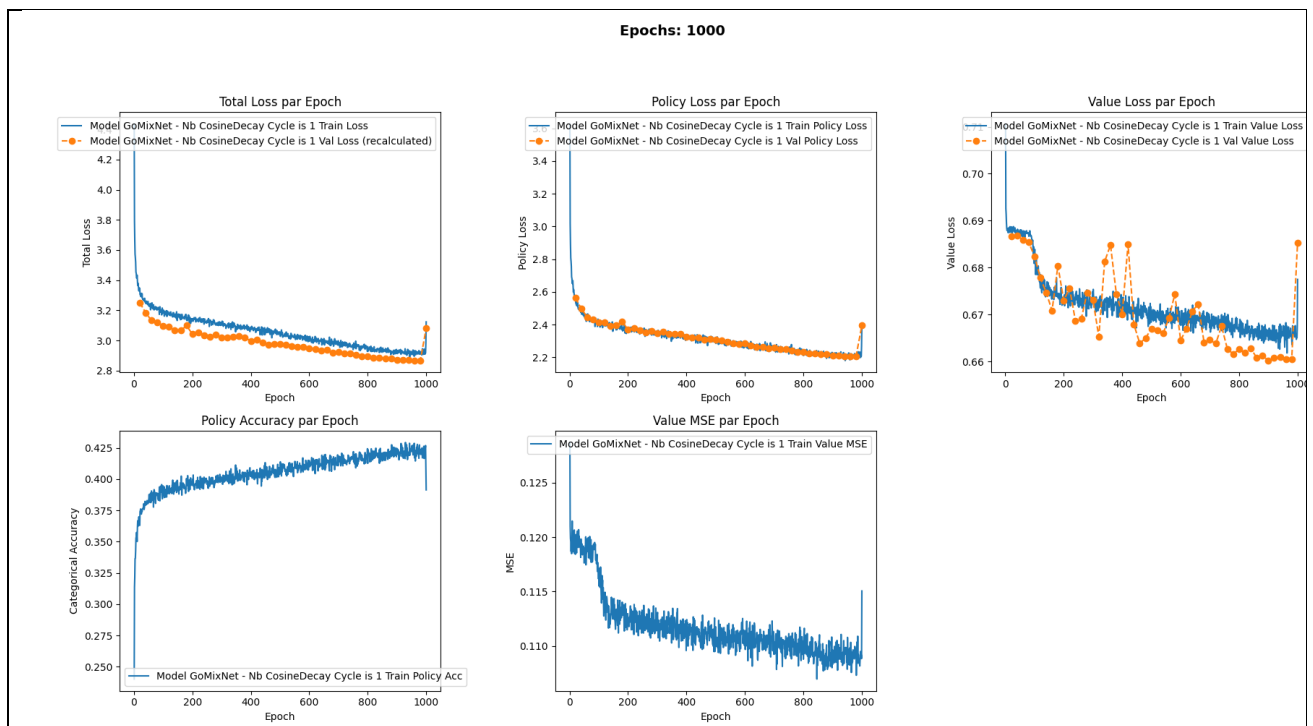


7.2.2 Test de la dimension de la taille des batchs d'entraînement



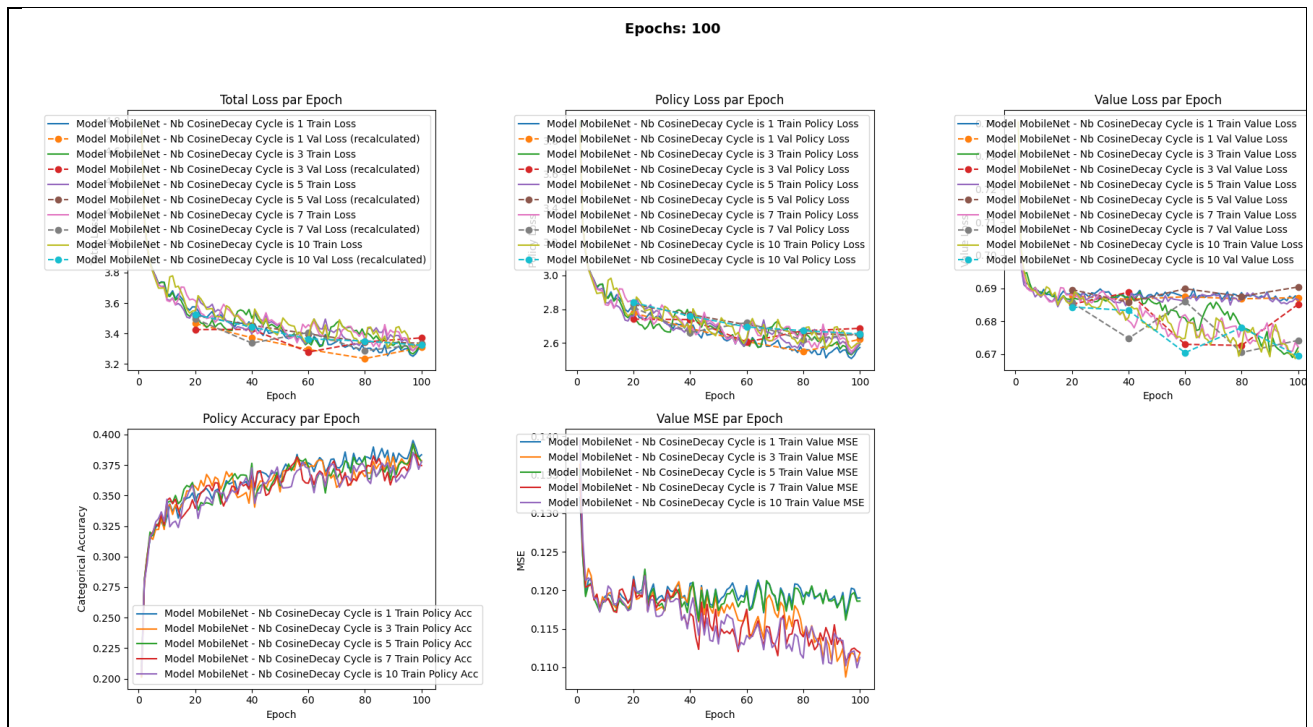


7.2.3 Test MixNet



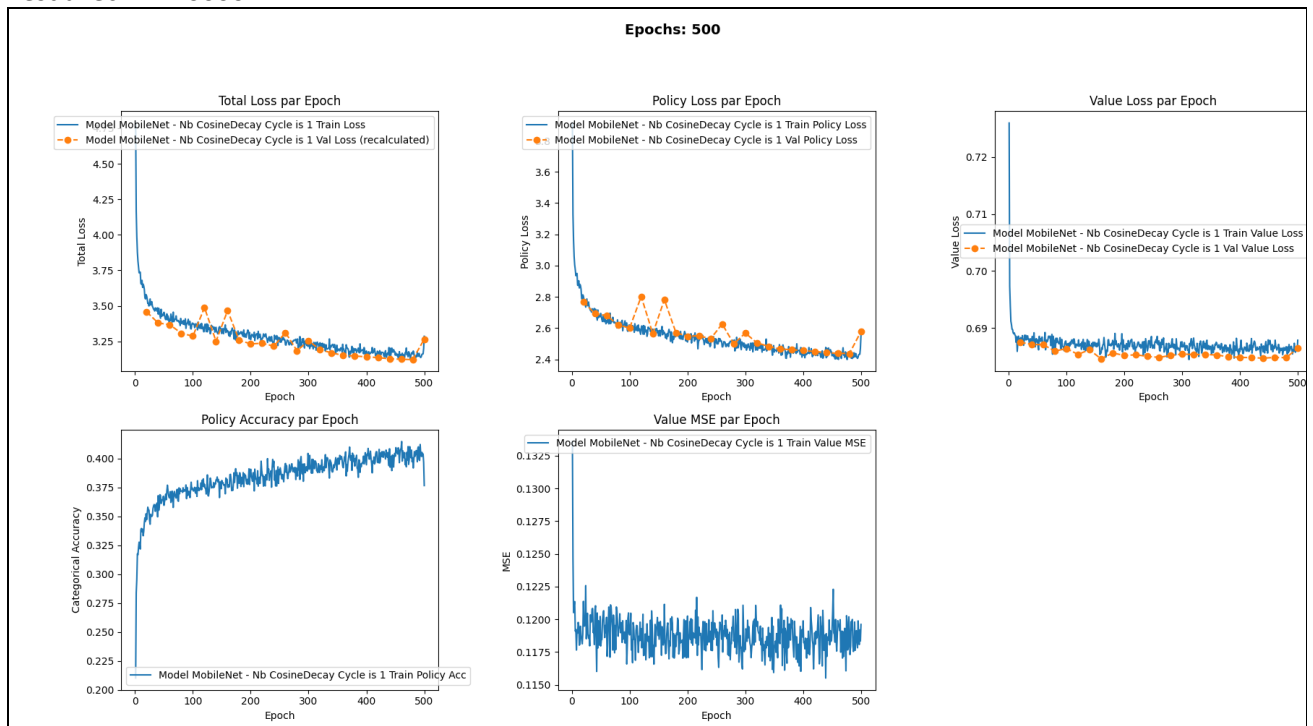
7.2.4 Test du learning rate et politique d'adaptation

7.2.5 Test du Consine Anneling avec redémarrage



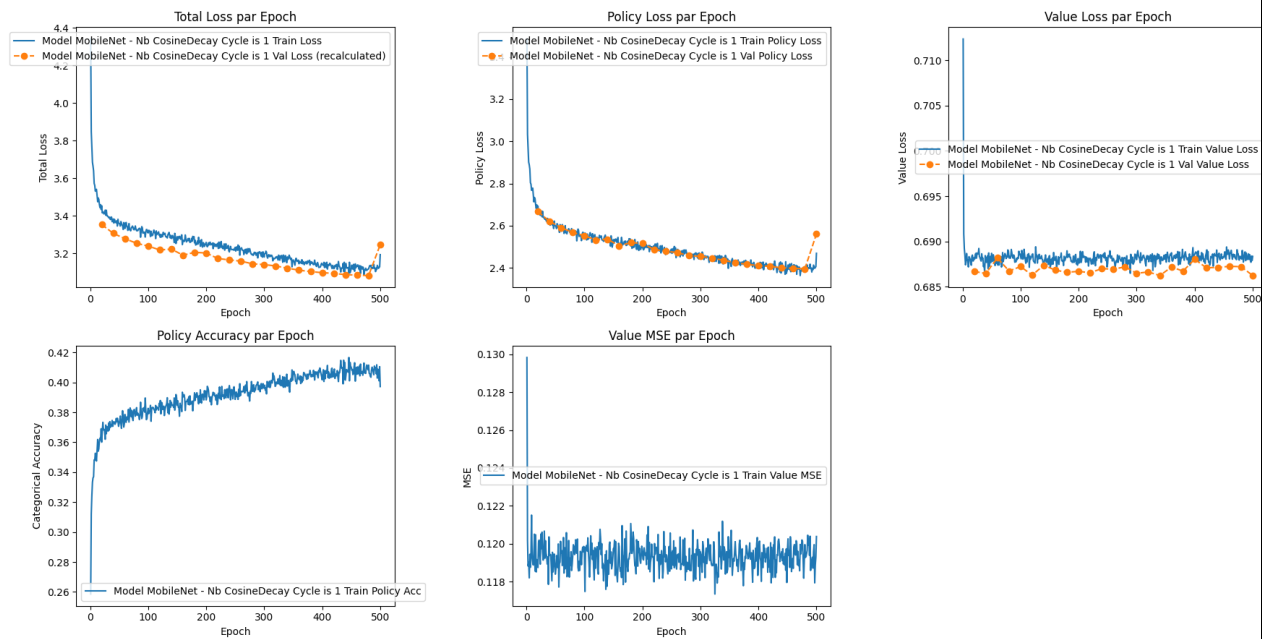
7.2.6 Test de la dimension des données entrainées pour une epoch

Test avec N = 10000



Test avec N=25000

Epochs: 500



8 Conclusions

9. Annexes

